

DEEP LEARNING — COMPLETE EVOLUTION

From Perceptron to Transformers

Consolidated Interview Question Bank — 51 Topics Across 7 Modules

Data Science & AI Engineer Interview Preparation

Modules A-G: Foundations | Training | CNNs | RNNs/LSTM/GRU | Attention | Transformers |
BERT/GPT/ViT/LLMs

Index — Topics Covered

MODULE A — Foundations: From Biological Neuron to Perceptron

1. Biological Neuron Inspiration & McCulloch-Pitts Model
2. Perceptron — Geometric Intuition & Learning Rule
3. Limitations of Perceptron — XOR Problem & Linear Separability
4. Multi-Layer Perceptron (MLP) — Solving Non-Linear Problems
5. Activation Functions: Sigmoid, Tanh, ReLU, Leaky ReLU, GELU, Swish

MODULE B — Training Neural Networks

6. Forward Propagation
7. Backpropagation — Chain Rule & Computational Graphs
8. Loss Functions for Deep Learning
9. Gradient Descent Variants: SGD, Mini-Batch, Momentum, Nesterov
10. Adaptive Optimizers: AdaGrad, RMSProp, Adam, AdamW
11. Vanishing & Exploding Gradients
12. Weight Initialization — Xavier & He Initialization
13. Batch Normalization & Layer Normalization
14. Dropout & Other Regularization for Neural Networks
15. Hyperparameter Tuning for Neural Networks

MODULE C — Convolutional Neural Networks (CNNs)

16. Why CNNs — Limitations of MLPs for Images
17. Convolution Operation, Filters/Kernels & Feature Maps
18. Padding, Stride & Pooling
19. CNN Architectures: LeNet, AlexNet, VGG
20. Inception / GoogLeNet — Multi-Scale Features
21. ResNet — Skip Connections & Solving Vanishing Gradients in Deep Nets
22. Transfer Learning with CNNs
23. Object Detection Basics (R-CNN Family, YOLO Overview)
24. Image Segmentation Basics (U-Net Overview)

MODULE D — Sequence Models: RNNs and Beyond

25. Why Sequence Models — Limitations of CNNs/MLPs for Sequential Data
26. Recurrent Neural Networks (RNN) — Architecture & Hidden State
27. Backpropagation Through Time (BPTT)
28. Vanishing/Exploding Gradients in RNNs (Long-Range Dependencies)
29. LSTM — Gates (Forget, Input, Output) & Cell State
30. GRU — Simplified Gating
31. Bidirectional RNNs
32. Sequence-to-Sequence (Seq2Seq) Models
33. Encoder-Decoder Architecture — Deeper Dive

MODULE E — Attention Mechanism: The Bridge to Transformers

34. Limitations of Seq2Seq — The Fixed Context Vector Bottleneck
35. Attention Mechanism — Intuition (Bahdanau/Luong Attention)
36. Attention as Weighted Average — Query, Key, Value Framing
37. Self-Attention — Relating All Positions to Each Other

MODULE F — Transformers

- 38. 'Attention is All You Need' — Motivation: Replacing Recurrence
- 39. Transformer Architecture Overview (Encoder-Decoder)
- 40. Multi-Head Self-Attention — Mathematical Formulation
- 41. Positional Encoding — Why and How
- 42. Feed-Forward Layers, Residual Connections & Layer Norm in Transformers
- 43. Masked Self-Attention (Decoder)
- 44. Transformer Training — Teacher Forcing (Revisited for Transformers)

MODULE G — Transformer-Based Models & Modern NLP

- 45. BERT — Masked Language Modeling & Bidirectional Context
- 46. GPT Family — Autoregressive Language Modeling, Decoder-Only
- 47. Encoder-Only vs Decoder-Only vs Encoder-Decoder (BERT vs GPT vs T5)
- 48. Fine-Tuning vs Pretraining — Transfer Learning in NLP
- 49. Tokenization for Transformers (BPE, WordPiece, SentencePiece)
- 50. Vision Transformers (ViT) — Applying Transformers to Images
- 51. Scaling Laws & Large Language Models (Overview)

MODULE A — Foundations: From Biological Neuron to Perceptron

Topic 1: Biological Neuron Inspiration & McCulloch-Pitts Model

□ Concept Overview

Artificial neurons are loosely inspired by biological neurons: dendrites receive signals, the cell body sums/processes them, the axon fires an output if a threshold is exceeded. The McCulloch-Pitts (1943) model formalized this: output = 1 if weighted sum of inputs $\geq$ threshold, else 0 — the first mathematical model of a neuron, predating the perceptron.

Question	Model Answer
What is the McCulloch-Pitts neuron model and why is it historically important?	A binary threshold unit: output = 1 if $\sum(w_i x_i) \geq \text{threshold}$, else 0, with fixed (non-learnable) weights. Historically important as the first mathematical abstraction of a neuron (1943) — established that simple threshold units could compute logical functions (AND, OR, NOT), laying conceptual groundwork for all later neural network models.
How does the biological neuron's structure map to an artificial neuron's computation?	Dendrites (receive signals from other neurons) $\rightarrow$ input features x_i with weights w_i . Cell body (integrates signals) $\rightarrow$ weighted sum + bias: $\sum(w_i x_i) + b$. Axon firing threshold $\rightarrow$ activation function (originally a step function, now sigmoid/ReLU etc). Synaptic strength $\rightarrow$ learnable weight w_i . The mapping is a loose analogy, not biologically precise, but motivated the framework.
What was the key limitation of the McCulloch-Pitts model compared to the perceptron?	McCulloch-Pitts neurons had FIXED, hand-designed weights and thresholds — no learning mechanism. The perceptron (Rosenblatt, 1958) introduced a LEARNING RULE — weights are adjusted automatically based on errors on training data. This shift from 'hand-crafted logic gates' to 'learned weights from data' is the conceptual birth of machine learning as we know it.

Topic 2: Perceptron — Geometric Intuition & Learning Rule

□ Concept Overview

The perceptron is a single linear unit: $\text{output} = \text{step}(w^T x + b)$. Geometrically, it's a hyperplane separating two classes. The perceptron learning rule: for each misclassified point, update $w \leftarrow w + \text{learning_rate} \cdot (y_{\text{true}} - y_{\text{pred}}) \cdot x$ — nudging the hyperplane toward correctly classifying that point. Guaranteed to converge IF the data is linearly separable (Perceptron Convergence Theorem).

Question	Model Answer
What is the perceptron and what does it compute geometrically?	A single linear unit: $\text{output} = \text{step}(w^T x + b)$, where $\text{step}(z) = 1$ if $z \geq 0$ else 0. Geometrically, $w^T x + b = 0$ defines a hyperplane — the perceptron classifies points based on which side of this hyperplane they fall on. It's the simplest possible linear binary classifier — same geometric picture as logistic regression's decision boundary, but with a hard step instead of a smooth sigmoid.
Describe the perceptron learning rule.	For each training example (x, y) : compute prediction $y_{\text{pred}} = \text{step}(w^T x + b)$. If correct ($y_{\text{pred}} = y$): no update. If incorrect: update $w := w + \text{lr} \cdot (y - y_{\text{pred}}) \cdot x$ and $b := b + \text{lr} \cdot (y - y_{\text{pred}})$. This nudges the hyperplane in the direction that would have correctly classified the misclassified point. Repeat over the dataset (epochs) until no misclassifications (if data is linearly separable).
What does the Perceptron Convergence Theorem state, and what is its key limitation?	If the training data is LINEARLY SEPARABLE, the perceptron learning rule is guaranteed to converge to a separating hyperplane in a finite number of steps. Key limitation: if the data is NOT linearly separable (e.g., XOR), the perceptron will NEVER converge — it will oscillate forever, since no hyperplane can perfectly separate the classes.

Topic 3: Limitations of Perceptron — XOR Problem & Linear Separability

□ Concept Overview

The XOR function (output=1 if inputs differ, 0 if same) is NOT linearly separable — no single line

can separate the (0,0)/(1,1) points from (0,1)/(1,0) points. Minsky & Papert's 1969 book highlighted this limitation, causing the first 'AI winter' for neural networks — until multi-layer networks (with non-linear hidden layers) were shown to solve XOR.

Question	Model Answer
Why can't a single perceptron solve the XOR problem?	XOR truth table: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Plotting these points: the two classes (output=0 and output=1) are arranged such that NO SINGLE STRAIGHT LINE can separate them — the '0' points are diagonal corners and the '1' points are the other diagonal. XOR is NOT linearly separable, so a single-layer perceptron (which can only represent a hyperplane/line) fundamentally cannot represent this function, regardless of how it's trained.
How does adding a hidden layer solve the XOR problem?	A hidden layer can learn intermediate non-linear representations: e.g., one hidden unit can compute (x1 OR x2), another can compute (x1 AND x2), and the output layer computes (hidden1 AND NOT hidden2) = XOR. Geometrically, the hidden layer TRANSFORMS the input space into a new space where the classes BECOME linearly separable — this is the core insight behind why depth (multiple layers) gives neural networks much greater representational power than a single layer.
What was the historical impact of the XOR limitation on the field of AI?	Minsky and Papert's 1969 book 'Perceptrons' rigorously proved single-layer perceptrons couldn't solve XOR (and other non-linearly-separable problems), which was widely (mis)interpreted as 'neural networks are fundamentally limited' — contributing to reduced funding and interest in neural network research, the first 'AI winter' (roughly 1970s-1980s). The eventual development of backpropagation for training MULTI-layer networks (1980s) resolved this and revived the field.

Topic 4: Multi-Layer Perceptron (MLP) — Solving Non-Linear Problems

□ Concept Overview

An MLP stacks multiple layers of neurons: input layer -> one or more hidden layers (with non-linear activation functions) -> output layer. The Universal Approximation Theorem states an MLP

with even ONE sufficiently wide hidden layer (and non-linear activation) can approximate ANY continuous function to arbitrary precision. Depth allows learning hierarchical features.

Question	Model Answer
What is a Multi-Layer Perceptron (MLP) and why does it overcome the perceptron's linear limitation?	An MLP is a stack of layers: each layer applies a LINEAR transformation (weighted sum) followed by a NON-LINEAR activation function. Stacking these creates compositions of non-linear functions — the overall function the network represents is highly non-linear, even though each individual layer is 'linear + simple non-linearity'. This compositional non-linearity is what allows MLPs to represent functions (like XOR) that a single linear layer cannot.
What is the Universal Approximation Theorem and what does it NOT tell you?	States: a feedforward network with a single hidden layer containing a FINITE (but possibly very large) number of neurons, with a non-linear activation function, can approximate ANY continuous function on a bounded domain to arbitrary precision. What it does NOT tell you: HOW MANY neurons are needed (could be exponentially many for complex functions), or HOW TO TRAIN such a network to actually find the right weights (the theorem is an EXISTENCE proof, not a constructive/learnability guarantee) — this is why DEEP (multi-layer) networks are used in practice, as depth can represent complex functions far more EFFICIENTLY (fewer total parameters) than a single very-wide layer.
Why is depth (multiple hidden layers) often more parameter-efficient than width (one very wide layer)?	Depth allows HIERARCHICAL FEATURE COMPOSITION — each layer builds on the representations of the previous layer (e.g., in vision: edges -> shapes -> object parts -> objects). A shallow-but-wide network must represent each pattern somewhat independently/redundantly, often requiring exponentially more neurons to achieve the same expressiveness as a deep network that REUSES and COMBINES simpler learned features across layers. This is the core motivation for 'deep' learning as opposed to just 'wide' learning.

Topic 5: Activation Functions: Sigmoid, Tanh, ReLU, Leaky ReLU, GELU, Swish

□ Concept Overview

Activation functions introduce non-linearity, enabling networks to learn complex functions. Sigmoid: squashes to (0,1), suffers vanishing gradients at extremes. Tanh: squashes to (-1,1), zero-centered but still saturates. ReLU: $\max(0,x)$, avoids saturation for $x>0$, computationally cheap, but 'dies' for $x<0$ (zero gradient). Leaky ReLU/GELU/Swish: modern variants addressing ReLU's dead-neuron problem with smoother/non-zero negative behavior.

Question	Model Answer
What is the vanishing gradient problem with Sigmoid and Tanh activations?	Both Sigmoid and Tanh SATURATE for large input — their derivative approaches ZERO in these regions (Sigmoid derivative max is 0.25 at $x=0$, approaches 0 for $ x >\sim 5$; Tanh similar but max derivative=1). During backpropagation, gradients are MULTIPLIED through each layer — if each layer's activation derivative is near zero, the gradient SHRINKS EXPONENTIALLY with depth, becoming effectively zero by the time it reaches early layers. This 'vanishing gradient' makes deep networks with Sigmoid/Tanh extremely slow/impossible to train.
Why did ReLU become the dominant activation function for deep networks, and what is its main weakness?	$\text{ReLU}(x)=\max(0,x)$. Advantages: derivative is exactly 1 for $x>0$ (NO saturation/vanishing gradient on the positive side), computationally trivial (just a threshold, no exponentials), induces sparsity (many neurons output exactly 0). Main weakness: 'dying ReLU' — if a neuron's input is consistently negative (e.g., due to a bad weight update), its gradient is PERMANENTLY ZERO for $x<0$, so it NEVER updates again — the neuron is 'dead' forever, regardless of future training.
How do Leaky ReLU, GELU, and Swish address ReLU's limitations?	Leaky ReLU: $f(x)=x$ if $x>0$, else $\alpha \cdot x$ (small α like 0.01) — gives a small non-zero gradient for $x<0$, preventing neurons from permanently dying. GELU (Gaussian Error Linear Unit): smooth, probabilistic version of ReLU — used in BERT/GPT, $x \cdot \Phi(x)$ where Φ is the Gaussian CDF — smooth everywhere, no hard cutoff. Swish: $x \cdot \text{sigmoid}(x)$ — smooth, self-gated, non-monotonic (small negative dip before $x=0$), found empirically to outperform ReLU in deep networks (used in EfficientNet).
[Advanced] Why do modern transformer architectures (BERT, GPT) typically use GELU instead of ReLU?	GELU is SMOOTH (infinitely differentiable) everywhere, unlike ReLU's sharp kink at $x=0$ — smoother loss landscapes can aid optimization in very deep/wide transformer networks. GELU's probabilistic interpretation (weighting input by the probability it would be 'kept' under a stochastic regularizer) provides a softer, more gradual transition that empirically improves performance

in large-scale language models. The marginal compute cost of GELU's smoothness is negligible relative to the overall transformer computation, so the empirical performance gain outweighs ReLU's simplicity advantage at this scale.

MODULE B — Training Neural Networks

Topic 6: Forward Propagation

□ Concept Overview

Forward propagation computes the network's output given an input: at each layer, compute $z = W \cdot x + b$ (linear transformation), then $a = \text{activation}(z)$ (non-linearity), pass a as input to the next layer. This produces the final prediction, which is compared to the true label via a loss function.

Question	Model Answer
What happens during forward propagation in a neural network?	Starting with input x , at each layer l : compute $z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}$ (linear combination of previous layer's activations), then $a^{(l)} = \text{activation}(z^{(l)})$ (apply non-linearity). $a^{(0)} = x$ (input). The final layer's output $a^{(L)}$ is the network's prediction. This is purely a sequence of matrix multiplications and element-wise non-linear functions — no learning happens here, just computing the current prediction.
Why do we need to STORE intermediate values (z and a at each layer) during forward propagation?	These intermediate values are REQUIRED for backpropagation — computing gradients via the chain rule requires the activation values and pre-activation (z) values at each layer (e.g., the derivative of ReLU depends on whether z was positive or negative; the gradient w.r.t. weights depends on the previous layer's activation a). Storing these during the forward pass (instead of recomputing them later) is a memory-for-speed tradeoff — this is why training deep networks requires more memory than just running inference.

Topic 7: Backpropagation — Chain Rule & Computational Graphs

□ Concept Overview

Backpropagation computes gradients of the loss with respect to every weight, using the CHAIN RULE applied layer-by-layer from output back to input. A computational graph represents the network as nodes (operations) and edges (data flow) — backprop is reverse-mode automatic

differentiation on this graph, efficiently computing all gradients in roughly the same time as one forward pass.

Question	Model Answer
What is backpropagation and what mathematical principle does it rely on?	Backpropagation computes the gradient of the loss function with respect to EVERY weight in the network, by applying the CHAIN RULE of calculus repeatedly, propagating gradients BACKWARD from the output layer to the input layer. For layer l : $dL/dW^{(l)}$ depends on $dL/da^{(l)}$ (gradient flowing back from later layers) multiplied by $da^{(l)}/dz^{(l)}$ (activation derivative) and $dz^{(l)}/dW^{(l)}$ ($=a^{(l-1)}$, the previous layer's activation).
What is a computational graph and how does it relate to backpropagation?	A computational graph represents the network's computation as a DAG (directed acyclic graph): nodes are operations (add, multiply, activation functions), edges represent data flow (tensors). Forward propagation = traversing the graph forward, computing values at each node. Backpropagation = traversing the graph BACKWARD, applying the chain rule at each node to accumulate gradients — this is 'reverse-mode automatic differentiation', the algorithm implemented by autograd in PyTorch/TensorFlow.
Why is backpropagation computationally efficient — what is its time complexity relative to forward propagation?	Backpropagation has roughly the SAME time complexity as forward propagation (a constant factor, typically 2-3x) — NOT exponential in the number of layers, despite computing gradients for potentially millions/billions of parameters. This efficiency comes from REUSING intermediate computations via the chain rule (dynamic-programming-like reuse) — without backprop, computing each parameter's gradient independently via finite differences would require a SEPARATE forward pass PER PARAMETER, making training deep networks computationally infeasible.

Topic 8: Loss Functions for Deep Learning

□ Concept Overview

MSE (Mean Squared Error): for regression, $L=(y-y_{\text{hat}})^2$ — penalizes large errors heavily.
Cross-entropy: for classification, $L=-\sum(y_i \cdot \log(y_{\text{hat}}_i))$ — measures the 'distance' between

predicted and true probability distributions, pairs naturally with softmax output. Choice of loss function shapes the gradient signal and must match the task (regression vs classification) and output activation.

Question	Model Answer
Why is cross-entropy loss preferred over MSE for classification tasks with a softmax output?	Cross-entropy combined with softmax produces a CLEAN gradient: $dL/dz = (y_hat - y)$ for the output layer — simple and well-behaved regardless of how confident/wrong the prediction is. MSE with softmax can produce VERY SMALL gradients when the prediction is confidently WRONG (softmax saturates near 0 or 1, and MSE's gradient involves a derivative term that shrinks in these saturated regions) — leading to slow learning exactly when the network most needs to correct itself. Cross-entropy avoids this 'gradient vanishing when most needed' problem.
What is the relationship between cross-entropy loss and Maximum Likelihood Estimation (MLE)?	Minimizing cross-entropy loss is EQUIVALENT to performing Maximum Likelihood Estimation under the assumption that the model outputs represent a probability distribution (via softmax/sigmoid) and the true labels are samples from the TRUE distribution. Cross-entropy = $-\log(\text{likelihood of the true label under the model's predicted distribution})$ — minimizing it = maximizing the likelihood the model assigns to the correct answers. This gives cross-entropy a principled statistical justification beyond just 'it works well empirically'.

Topic 9: Gradient Descent Variants: SGD, Mini-Batch, Momentum, Nesterov

□ Concept Overview

Batch GD: compute gradient over ENTIRE dataset per update — accurate but slow/memory-heavy. SGD: update using ONE sample at a time — fast but noisy. Mini-batch GD: compromise (e.g., 32-256 samples) — standard practice. Momentum: accumulate a velocity term (exponential moving average of past gradients) to smooth updates and accelerate through consistent-direction regions. Nesterov: 'look-ahead' momentum — compute gradient at the anticipated future position.

Question	Model Answer
What is the tradeoff between Batch Gradient Descent, SGD, and Mini-Batch Gradient	Batch GD: gradient computed over the FULL dataset — accurate gradient direction but

Descent?	extremely slow per update (one update per epoch) and requires the entire dataset in memory. SGD: gradient from ONE random sample — very fast/noisy updates, the noise can actually help escape shallow local minima/saddle points but convergence is erratic. Mini-batch (typically 32-512 samples): balances both — reasonably accurate gradient estimate, fits in GPU memory, allows VECTORIZED/parallel computation (GPU efficiency), and the moderate noise still provides some regularization benefit. Standard practice in essentially all modern deep learning.
What problem does Momentum solve and how does it work?	Vanilla SGD/mini-batch GD can oscillate inefficiently in narrow valleys of the loss landscape (e.g., steep in one direction, shallow in another) — zig-zagging slowly toward the minimum. Momentum maintains a VELOCITY term $v = \beta \cdot v_{\text{prev}} + \text{gradient}$ (exponential moving average of past gradients), and updates weights using v instead of the raw gradient. This DAMPENS oscillations in directions where gradients frequently flip sign (they cancel out in the average) while ACCELERATING in directions where gradients consistently point the same way (they accumulate) — like a ball rolling downhill gaining speed in a consistent direction.
How does Nesterov Accelerated Gradient (NAG) differ from standard Momentum?	Standard Momentum: compute gradient at the CURRENT position, then apply the momentum-adjusted step. Nesterov: first take a 'look-ahead' step using the CURRENT momentum/velocity (anticipating where you're about to go), THEN compute the gradient AT THAT LOOK-AHEAD POSITION, and use THIS gradient to correct the update. This 'look before you leap' approach provides a form of CORRECTION — if the momentum is about to overshoot the minimum, the look-ahead gradient (computed past the minimum) will point BACKWARD, partially counteracting the overshoot, giving faster and more stable convergence than vanilla momentum.

Topic 10: Adaptive Optimizers: AdaGrad, RMSProp, Adam, AdamW

□ Concept Overview

These optimizers adapt the **LEARNING RATE PER PARAMETER** based on historical gradient information. AdaGrad: divides learning rate by $\sqrt{\text{sum of squared past gradients}}$ — learning rate shrinks monotonically (can become too small). RMSProp: uses an **EXPONENTIAL MOVING AVERAGE** of squared gradients instead of a sum — fixes AdaGrad's diminishing learning rate. Adam: combines Momentum (first moment) + RMSProp (second moment) — the de facto default optimizer for deep learning. AdamW: decouples weight decay from the adaptive gradient update, fixing a subtle bug in Adam's L2 regularization.

Question	Model Answer
What problem does AdaGrad solve, and what is its main limitation?	AdaGrad gives EACH PARAMETER its own adaptive learning rate, scaled inversely by $\sqrt{\text{sum of all past squared gradients for that parameter}}$. Parameters with consistently LARGE gradients get their effective learning rate SHRUNK MORE ; parameters with small/infrequent gradients (e.g., rare-feature embeddings) retain LARGER effective learning rates — useful for sparse data (NLP). Limitation: the sum of squared gradients only GROWS over time, so the effective learning rate MONOTONICALLY DECREASES and can become VANISHINGLY SMALL long before training converges — training effectively 'stalls'.
How does RMSProp fix AdaGrad's diminishing learning rate problem?	RMSProp replaces AdaGrad's SUM of all past squared gradients with an EXPONENTIAL MOVING AVERAGE (EMA) : $E[g^2]_t = \beta * E[g^2]_{t-1} + (1-\beta) * g_t^2$. This EMA 'forgets' old gradients (weighted by β , typically 0.9/0.99) — so the denominator doesn't grow unboundedly, and the effective learning rate can stay at a useful, ROUGHLY STABLE magnitude throughout training, rather than decaying to zero.
What does Adam combine, and what do its two main hyperparameters (β_1 , β_2) control?	Adam = Momentum (tracks an EMA of gradients — the 'first moment', controlled by β_1 , typically 0.9) + RMSProp (tracks an EMA of SQUARED gradients — the 'second moment', controlled by β_2 , typically 0.999). The first moment provides the directional momentum/smoothing; the second moment provides the per-parameter adaptive learning rate scaling. Adam also includes a BIAS CORRECTION term to account for the EMAs being initialized at zero (which biases early estimates toward zero) — without this correction, early training steps would have artificially small effective updates.
[Advanced] What problem does AdamW fix compared to standard Adam with L2 regularization?	In standard Adam, L2 regularization is implemented by adding $\lambda * w$ to the GRADIENT before the adaptive scaling — but this means the weight decay term ALSO gets divided by Adam's adaptive denominator ($\sqrt{\text{of}}$

second moment), causing the EFFECTIVE weight decay to vary unpredictably PER PARAMETER based on their gradient history — parameters with large historical gradients get LESS weight decay than intended. AdamW DECOUPLES weight decay from the gradient-based update: it applies $w := w - lr * \text{adam_update}$ AND SEPARATELY $w := w * (1 - lr * \text{weight_decay})$ — ensuring weight decay is applied UNIFORMLY/correctly regardless of the adaptive learning rate scaling. AdamW is now the standard choice for training transformers (BERT, GPT).

Topic 11: Vanishing & Exploding Gradients

□ Concept Overview

In deep networks, gradients are PRODUCTS of many terms (one per layer, via chain rule). If these terms are consistently < 1 (e.g., from saturating activations), the product shrinks exponentially toward zero with depth — vanishing gradients (early layers barely update). If terms are consistently > 1 , the product grows exponentially — exploding gradients (unstable, NaN losses). Both are fundamental obstacles to training deep networks, motivating ReLU, normalization, residual connections, and gradient clipping.

Question	Model Answer
What causes vanishing gradients in deep networks, and what are the symptoms during training?	Caused by: backpropagation multiplies many per-layer derivative terms together (chain rule across L layers). If each term has magnitude < 1 (common with Sigmoid/Tanh activations whose derivatives are ≤ 0.25 , or poorly-scaled weight initialization), the product shrinks EXPONENTIALLY with depth (e.g., $0.25^{20} \approx 10^{-12}$). Symptoms: early/lower layers' weights barely change during training (their gradients are near-zero), loss plateaus early, deeper networks perform WORSE than shallower ones (the opposite of what depth should provide).
What causes exploding gradients, and what is a common fix?	Caused by: per-layer derivative terms with magnitude > 1 (e.g., large weight initializations, or recurrent architectures where the same weight matrix is applied repeatedly over many time steps) — the product GROWS exponentially with depth/sequence length. Symptoms: loss suddenly becomes NaN/infinity, weight values explode to huge magnitudes, training becomes unstable/diverges. Common fix: GRADIENT

	CLIPPING — if the gradient's norm exceeds a threshold, RESCALE it down to that threshold before applying the update — caps the maximum step size without changing the gradient's DIRECTION.
How do ReLU activations, proper weight initialization, batch normalization, and residual connections each help mitigate vanishing gradients?	ReLU: derivative is exactly 1 (not <1) for positive inputs — doesn't shrink gradients through that path. Proper initialization (Xavier/He): scales initial weights so that activations/gradients maintain roughly CONSTANT VARIANCE across layers at the START of training, preventing immediate vanishing/exploding. Batch Normalization: re-normalizes activations at each layer to have stable mean/variance throughout training, keeping gradients in a well-behaved range. Residual connections (ResNet): provide a direct 'gradient highway' — the SKIP CONNECTION allows gradients to flow directly to earlier layers WITHOUT being multiplied through all intermediate layer derivatives, bypassing the vanishing problem entirely for that path.

Topic 12: Weight Initialization — Xavier & He Initialization

□ Concept Overview

Initializing weights too small causes activations/gradients to shrink toward zero through layers (vanishing); too large causes them to grow (exploding). Xavier/Glorot initialization: scales weights based on layer size (fan-in/fan-out) to keep variance roughly constant — designed for Sigmoid/Tanh. He initialization: similar but accounts for ReLU zeroing out half of activations, using a larger scale factor — designed for ReLU networks.

Question	Model Answer
Why does weight initialization matter, and what happens with naive initialization (e.g., all zeros, or very large random values)?	All-zeros initialization: every neuron in a layer computes the IDENTICAL output and receives the IDENTICAL gradient (since all weights are the same) — they remain identical FOREVER (symmetry never breaks), effectively collapsing the layer to a single neuron's worth of capacity, regardless of layer width. Very large random values: activations grow huge through successive layers (exploding), often saturating activation functions immediately (e.g., Sigmoid outputs near 0 or 1 everywhere) — gradients vanish from the START of training, before any learning can occur.
What is Xavier (Glorot) initialization and what is it	Initializes weights from a distribution with

designed to achieve?	variance = $1/\text{fan_in}$ (or $2/(\text{fan_in}+\text{fan_out})$ for the normalized version), where fan_in = number of input connections to the layer. Goal: keep the VARIANCE of activations (forward pass) AND gradients (backward pass) roughly CONSTANT across layers — preventing the exponential shrink/growth that causes vanishing/exploding. Designed for SYMMETRIC, roughly-linear-near-zero activations like Sigmoid/Tanh, where this variance-preservation analysis holds.
Why does He initialization use a different scaling factor than Xavier, and when should each be used?	He initialization uses variance = $2/\text{fan_in}$ (double Xavier's scale). Reason: ReLU zeros out approximately HALF of its inputs (all negative values become 0) — this effectively HALVES the variance of activations passing through a ReLU layer compared to a linear/Tanh layer. He initialization's 2x factor compensates for this halving, restoring the variance-preserving property for ReLU-based networks. Rule of thumb: use He initialization for ReLU/Leaky ReLU networks (the vast majority of modern CNNs/MLPs), Xavier for Sigmoid/Tanh networks.

Topic 13: Batch Normalization & Layer Normalization

□ Concept Overview

Batch Norm: normalizes activations across the BATCH dimension (per feature, using batch mean/variance) at each layer, then applies learnable scale/shift — stabilizes and speeds up training, allows higher learning rates, has regularization effect. Layer Norm: normalizes across the FEATURE dimension (per sample, not per batch) — works with batch size 1 and variable-length sequences, hence used in transformers/RNNs instead of BatchNorm.

Question	Model Answer
What does Batch Normalization do and what problem does it solve?	For each mini-batch, BatchNorm normalizes each feature/activation to have mean=0, variance=1 ACROSS THE BATCH, then applies a learnable scale (gamma) and shift (beta): $y = \text{gamma} * ((x - \text{batch_mean}) / \sqrt{\text{batch_var} + \text{eps}}) + \text{beta}$. Solves 'internal covariate shift' — without it, the DISTRIBUTION of each layer's inputs changes as the previous layers' weights update during training, forcing each layer to continuously re-adapt to a shifting input distribution. BatchNorm stabilizes these distributions, allowing HIGHER learning rates, FASTER convergence, and providing a mild regularization effect (batch

	statistics add noise).
Why doesn't Batch Normalization work well for very small batch sizes or for RNNs/Transformers, and what alternative is used instead?	BatchNorm computes statistics ACROSS THE BATCH — with very small batches (e.g., batch size 1, common when memory-constrained for large models), the batch mean/variance estimates become extremely NOISY/unreliable. For RNNs/Transformers: sequences have VARIABLE LENGTH, and normalizing across the batch dimension at each time step mixes statistics from different positions/sequences in awkward ways. LAYER NORMALIZATION instead normalizes ACROSS THE FEATURE dimension FOR EACH INDIVIDUAL SAMPLE (independent of batch size and other samples) — this works identically regardless of batch size, and is the standard choice in transformers (BERT, GPT) and RNNs/LSTMs.
[Advanced] At inference/test time, how does Batch Normalization behave differently than during training?	During TRAINING: BatchNorm uses the CURRENT BATCH's mean and variance for normalization (and also maintains a running/exponential-moving-average estimate of these statistics across all batches seen). During INFERENCE: there may be no 'batch' (e.g., single-sample prediction), and using a single sample's statistics would be meaningless/unstable. So at inference, BatchNorm uses the STORED RUNNING AVERAGE statistics (computed during training) instead of computing fresh statistics from the (possibly single-sample) inference batch. This train/inference mismatch is a common source of subtle bugs if not handled correctly (e.g., forgetting to call <code>model.eval()</code> in PyTorch, which switches BatchNorm to use running statistics).

Topic 14: Dropout & Other Regularization for Neural Networks

□ Concept Overview

Dropout: randomly 'turns off' (zeros) a fraction p of neurons during EACH training step — forces the network to not rely on any single neuron, acts like training an ensemble of sub-networks. At test time, dropout is disabled (all neurons active), often with weight scaling. Other regularization: L1/L2 weight penalties (same as in classical ML), early stopping (stop training when validation loss starts increasing), data augmentation.

Question	Model Answer
How does Dropout work and why does it act as a regularizer?	During each training forward pass, each neuron is INDEPENDENTLY 'dropped' (output set to 0) with probability p (e.g., $p=0.5$), and the remaining active neurons' outputs are typically scaled by $1/(1-p)$ to maintain expected magnitude. This prevents the network from RELYING TOO HEAVILY on any specific neuron or specific combinations of neurons (co-adaptation) — each neuron must learn features that are USEFUL IN COMBINATION WITH MANY DIFFERENT RANDOM SUBSETS of other neurons, similar to training an implicit ENSEMBLE of many sub-networks that share weights, and averaging their predictions (approximately) at test time.
How does Dropout behave differently at training time vs inference time?	Training: randomly zero out neurons with probability p each forward pass (different random subset each time), scale remaining activations by $1/(1-p)$ ('inverted dropout', standard in modern frameworks). Inference: dropout is DISABLED — ALL neurons are active, with NO scaling needed (since inverted dropout already accounted for the expected scaling during training). Forgetting to disable dropout at inference (e.g., not calling <code>model.eval()</code> in PyTorch) causes inconsistent/non-deterministic predictions and degraded accuracy.
Compare early stopping, L2 regularization, and data augmentation as regularization techniques for deep learning.	Early stopping: monitor validation loss during training, STOP when it starts increasing (even though training loss continues decreasing) — directly prevents overfitting by limiting effective training time/model complexity, requires no architecture changes. L2 (weight decay): penalizes large weights, encourages smoother/simpler functions — same mechanism as Ridge regression, now typically implemented via AdamW's decoupled weight decay. Data augmentation: artificially expands the effective training set (random crops/flips/rotations for images, synonym replacement for text) — addresses overfitting at its ROOT CAUSE (insufficient data diversity) rather than constraining the model — often the SINGLE MOST EFFECTIVE regularization technique for vision tasks.

Topic 15: Hyperparameter Tuning for Neural Networks

□ Concept Overview

Key DL hyperparameters: learning rate (most important — too high diverges, too low is slow), batch size (affects gradient noise and GPU utilization), number of layers/neurons (model capacity), dropout rate, weight decay, optimizer choice, learning rate SCHEDULE (warmup, decay). Tuning approaches: learning rate finder (try increasing LR, find where loss starts diverging), random/grid search, learning rate schedules (cosine annealing, step decay, warmup).

Question	Model Answer
Why is the learning rate considered the most important hyperparameter in deep learning?	Too HIGH: loss oscillates wildly or DIVERGES (overshoots minima repeatedly, can become NaN). Too LOW: training is extremely SLOW, may get stuck in poor local minima/plateaus, and may not converge within a practical time budget. Unlike many other hyperparameters where a 'reasonable' value works adequately across a range, learning rate has a relatively NARROW 'sweet spot' that varies by model/dataset/optimizer — getting it badly wrong can make training completely fail (vs other hyperparameters that might just be 'somewhat suboptimal').
What is a learning rate WARMUP and why is it used, especially for transformers?	Warmup: start training with a VERY SMALL learning rate, gradually INCREASE it to the target learning rate over the first several hundred/thousand steps, THEN apply the normal schedule (e.g., decay). Reason: at the very start of training, weights are randomly initialized — gradients can be large/noisy/unrepresentative of the 'true' loss landscape, and adaptive optimizers (Adam) have poorly-calibrated second-moment estimates (near-zero EMA initialization) early on. A large learning rate applied immediately can cause early DIVERGENCE or push the model into a poor region it struggles to escape. Warmup gives the optimizer 'time to calibrate' before applying full-strength updates — essential for stable training of transformers (BERT, GPT) which are notoriously sensitive to early training dynamics.
What is a learning rate schedule, and compare step decay vs cosine annealing.	A learning rate schedule changes the learning rate OVER THE COURSE OF TRAINING (rather than keeping it constant) — typically starting higher (faster initial progress) and decreasing (finer convergence near the end). Step decay: multiply LR by a factor (e.g., 0.1) at predetermined epochs (e.g., epochs 30, 60, 90) — simple, but requires manually choosing WHEN to decay (sensitive to total training duration). Cosine annealing: LR follows a smooth cosine curve from initial value down to near-zero over the training duration — no manual 'decay points'

	needed, smooth transitions tend to produce slightly better final performance, and is the standard choice in most modern training recipes (often combined with warmup).
--	--

MODULE C — Convolutional Neural Networks (CNNs)

Topic 16: Why CNNs — Limitations of MLPs for Images

□ Concept Overview

An MLP applied to images flattens pixels into a 1D vector — destroying spatial structure (a 28x28 image becomes 784 independent numbers, losing the fact that nearby pixels are related). Also, MLPs require a SEPARATE weight for every pixel-to-neuron connection — for a 224x224 image, the first layer alone could have millions of parameters, making MLPs impractical and prone to overfitting for images.

Question	Model Answer
Why is flattening an image into a 1D vector for an MLP problematic?	Flattening DESTROYS SPATIAL STRUCTURE — pixel (0,0) and pixel (0,1) (adjacent, likely similar/related) end up as far-apart elements in the flattened vector with NO inherent relationship encoded, while pixel (0,0) and pixel (27,27) (far apart in the image) might end up adjacent in the vector. The MLP must LEARN spatial relationships from scratch with no structural prior — extremely data-inefficient, since the same pattern (e.g., an edge) appearing at different image locations must be learned SEPARATELY for each location.
Why does an MLP's parameter count become impractical for images, and how does this lead to overfitting?	A fully-connected layer connecting a 224x224x3 image (150,528 inputs) to even a modest 1000-neuron hidden layer requires $150,528 \times 1000 \approx 150$ MILLION weights — for ONE layer. This enormous parameter count: (1) requires massive amounts of training data to avoid overfitting (more parameters than typical dataset sizes), (2) is computationally expensive, (3) doesn't generalize across spatial positions — a cat in the top-left vs bottom-right of an image would need to be learned as essentially DIFFERENT patterns by a fully-connected layer.
What two key properties of convolutional layers directly address these MLP limitations?	1) PARAMETER SHARING : the same small filter (e.g., 3x3 weights) is applied across ALL spatial locations of the image — a feature (edge, corner) learned in one location is automatically detected everywhere, drastically reducing parameters and enabling TRANSLATION INVARIANCE . 2) LOCAL CONNECTIVITY : each output depends only on a small local neighborhood of the input (the filter's receptive field), not the entire image

— respects the natural prior that nearby pixels are more related than distant ones, and is far more parameter-efficient.

Topic 17: Convolution Operation, Filters/Kernels & Feature Maps

□ Concept Overview

A convolution slides a small filter/kernel (e.g., 3x3 matrix of learnable weights) across the input, computing a dot product at each position — producing a feature map. Different filters detect different patterns (edges, textures). Multiple filters per layer produce multiple feature maps (channels). Early layers detect simple patterns (edges); deeper layers combine these into complex patterns (shapes, objects).

Question	Model Answer
What is a convolution operation and what does a filter/kernel represent?	A filter/kernel is a small matrix of LEARNABLE WEIGHTS (e.g., 3x3 or 5x5). The convolution operation slides this filter across the input image, and at EACH position computes the ELEMENT-WISE PRODUCT between the filter and the corresponding image patch, then SUMS these products to produce a single output value. Repeating across all positions produces a FEATURE MAP — each value indicates how strongly the pattern encoded by the filter is present at that location. A filter is essentially a 'pattern detector' — its weights are LEARNED during training to detect useful patterns (edges, colors, textures).
What is the role of multiple filters/channels in a convolutional layer?	A single convolutional layer typically applies MANY different filters (e.g., 64 filters), each producing its OWN feature map — these are stacked to form a multi-channel output (e.g., 64 channels). Each filter learns to detect a DIFFERENT pattern (one might detect horizontal edges, another vertical edges, another a specific color blob). The next layer's filters then operate on this multi-channel input, combining these simple patterns into MORE COMPLEX patterns — this hierarchical combination across layers is how CNNs build up from edges to textures to object parts to whole objects.
[Advanced] What is the receptive field of a neuron, and how does it grow with network	The receptive field is the REGION OF THE ORIGINAL INPUT IMAGE that influences a given

depth?	neuron's value. A neuron in the FIRST conv layer (with a 3x3 filter) has a receptive field of 3x3 pixels. A neuron in the SECOND layer (also 3x3 filter), whose input is the first layer's output, has a receptive field of 5x5 in the ORIGINAL image (since each of its 3x3 inputs already 'sees' a 3x3 region). Receptive field GROWS with depth — deeper layers 'see' progressively larger regions of the original image, allowing them to detect larger-scale patterns (whole objects) while early layers detect small local patterns (edges). Pooling/strided convolutions accelerate this growth.
--------	---

Topic 18: Padding, Stride & Pooling

□ Concept Overview

Padding: add zeros around input borders so the filter can be applied at edge positions, controlling output size ('valid' = no padding, shrinks; 'same' = pad to preserve size). Stride: step size of the filter's sliding window — stride=1 moves one pixel at a time (dense), stride=2 skips every other position (downsamples by 2x). Pooling (max/average): downsamples feature maps by taking the max/average over small windows — reduces spatial dimensions, provides translation invariance, reduces computation.

Question	Model Answer
What is padding in CNNs and what is the difference between 'valid' and 'same' padding?	Padding adds extra pixels (typically zeros) around the border of the input before applying the convolution. 'Valid' padding: NO padding added — the output is SMALLER than the input (filter can't extend beyond the original borders, so fewer valid positions). 'Same' padding: pad such that the output has the SAME spatial dimensions as the input — necessary when stacking many conv layers without wanting the feature map to shrink to nothing, and ensures edge pixels are processed with roughly the same 'attention' as center pixels (without padding, edge pixels are used in FEWER filter applications than center pixels).
What does stride control, and what is the effect of stride=2 vs stride=1?	Stride is the step size the filter moves between applications. Stride=1: filter moves one pixel at a time — output has roughly the same spatial size as input (with same padding) — DENSE computation, preserves spatial resolution. Stride=2: filter skips every other position — output is approximately HALF the spatial size in

	each dimension — effectively DOWNSAMPLES the feature map by 2x, reducing computation for subsequent layers and increasing the receptive field growth rate. Strided convolutions are sometimes used INSTEAD of pooling for downsampling.
What is pooling and what are its main purposes?	Pooling (max pooling most common, also average pooling) downsamples a feature map by taking the MAX (or average) value within small non-overlapping windows (e.g., 2x2), reducing spatial dimensions by that factor. Purposes: 1) REDUCES COMPUTATION for subsequent layers (smaller feature maps = fewer operations). 2) Provides a degree of TRANSLATION INVARIANCE — a small shift in the input's exact pixel position may not change which value is the 'max' in a window, so the pooled output is relatively unchanged. 3) Progressively REDUCES SPATIAL DIMENSIONS toward the final classification layer, which needs a FIXED-SIZE representation regardless of input image size (when combined with adaptive pooling).

Topic 19: CNN Architectures: LeNet, AlexNet, VGG

□ Concept Overview

LeNet (1998): pioneering CNN for digit recognition, alternating conv+pooling layers, small (few layers, few filters). AlexNet (2012): won ImageNet, dramatically deeper/wider than LeNet, introduced ReLU and Dropout at scale, demonstrated GPUs could train large CNNs — sparked the deep learning revolution. VGG (2014): very deep (16-19 layers) using ONLY small 3x3 filters stacked repeatedly — showed that DEPTH with simple, uniform building blocks was key to performance.

Question	Model Answer
What was LeNet and what was its historical significance?	LeNet-5 (LeCun, 1998): one of the first successful CNNs, designed for handwritten digit recognition (MNIST) — alternating convolutional and pooling layers followed by fully-connected layers, using Tanh/Sigmoid activations (ReLU wasn't yet standard). Historical significance: demonstrated that CNNs could learn hierarchical visual features end-to-end via backpropagation and outperform hand-crafted feature approaches for digit recognition — but the field lacked the COMPUTE (GPUs) and DATA (large labeled datasets) to scale this approach for over a

	decade.
What made AlexNet a breakthrough, and why is 2012 considered a turning point for deep learning?	AlexNet (Krizhevsky et al., 2012) won the ImageNet competition by a HUGE margin over previous (non-deep-learning) approaches. Key innovations: 1) Used ReLU activations (instead of Tanh/Sigmoid) — enabled training MUCH DEEPER networks without vanishing gradients. 2) Used Dropout for regularization on a large model. 3) Trained on GPUs — made training a network with 60 million parameters on 1.2 million images COMPUTATIONALLY FEASIBLE for the first time. 2012 is considered a turning point because AlexNet's dramatic performance jump convinced the broader research/industry community that deep learning (with sufficient data and compute) could DRAMATICALLY outperform classical computer vision approaches — triggering the modern deep learning era.
What was VGG's key architectural insight, and what was its main drawback?	VGG (2014) showed that STACKING MANY SMALL (3x3) CONVOLUTIONAL FILTERS, with a SIMPLE, UNIFORM, REPEATED structure, and going VERY DEEP (16-19 layers), achieved excellent performance — simplicity and depth mattered more than complex hand-designed filter sizes. Insight: two stacked 3x3 conv layers have the same RECEPTIVE FIELD as one 5x5 layer but with FEWER PARAMETERS and an EXTRA NON-LINEARITY (more expressive). Main drawback: VGG has an enormous number of parameters (~138 million for VGG-16), mostly in the fully-connected layers near the output — large model size, slow inference, high memory usage — motivating later architectures (Inception, ResNet) to be more parameter-efficient.

Topic 20: Inception / GoogLeNet — Multi-Scale Features

□ Concept Overview

Inception modules apply MULTIPLE filter sizes (1x1, 3x3, 5x5) and pooling IN PARALLEL within the same layer, then concatenate their outputs — capturing features at multiple scales simultaneously without needing to choose one filter size. 1x1 convolutions are used for dimensionality reduction (reducing channel depth before expensive 3x3/5x5 convolutions), dramatically reducing computation while maintaining expressiveness.

Question

Model Answer

What is the core idea of an Inception module, and what problem does it solve?	Instead of choosing a SINGLE filter size (3x3 OR 5x5) for a layer (a design choice that's hard to get right — different objects/features appear at different SCALES in an image), an Inception module applies MULTIPLE filter sizes (1x1, 3x3, 5x5) AND pooling IN PARALLEL on the SAME input, then CONCATENATES all their outputs along the channel dimension. This lets the network learn to use whichever scale(s) are most useful for each pattern, rather than committing to one scale upfront — capturing multi-scale features within a single layer.
What is the role of 1x1 convolutions in the Inception architecture?	1x1 convolutions (applied BEFORE the more expensive 3x3/5x5 convolutions within an Inception module) perform DIMENSIONALITY REDUCTION along the CHANNEL dimension — e.g., reducing 256 channels down to 64 channels — WITHOUT affecting the SPATIAL dimensions. This dramatically reduces the COMPUTATIONAL COST of the subsequent larger filters (a 3x3 conv on 64 channels is far cheaper than on 256 channels) while still allowing the network to LEARN which channel combinations/information to preserve in the reduction — sometimes called a 'bottleneck' layer.

Topic 21: ResNet — Skip Connections & Solving Vanishing Gradients in Deep Nets

□ Concept Overview

ResNet introduced RESIDUAL/SKIP CONNECTIONS: $\text{output} = F(x) + x$, where $F(x)$ is a small block of layers and x is the block's input, added directly to the output via an 'identity shortcut'. This allows gradients to flow DIRECTLY through the skip connection to earlier layers, bypassing the vanishing gradient problem in very deep networks (100+ layers). Conceptually, each block only needs to learn the RESIDUAL (difference) from the identity mapping, which is often easier than learning the full transformation.

Question	Model Answer
What is a residual/skip connection and what problem does it solve?	A residual connection adds the INPUT of a block DIRECTLY to its OUTPUT: $\text{output} = F(x) + x$, where $F(x)$ represents the transformation learned by the block's layers (conv, batchnorm, activation). This solves the DEGRADATION PROBLEM in very deep networks: without skip connections, adding more layers BEYOND a

	certain depth made performance WORSE (not just diminishing returns, but actively worse) — because deep networks struggled to even learn the IDENTITY mapping ($F(x)=0$ such that $\text{output}=x$) when that would have been optimal, due to vanishing gradients and optimization difficulty. With the skip connection, if $F(x)=0$ is optimal, the network trivially achieves $\text{output}=x$ — making it EASY for the network to 'default to identity' and only deviate (learn $F(x) \neq 0$) where beneficial.
How do skip connections specifically address vanishing gradients during backpropagation?	During backpropagation, the gradient w.r.t. the block's input x is: $dL/dx = dL/d(\text{output}) * (dF/dx + 1)$ — the '+1' comes from the identity/skip path's derivative ($d(x)/dx=1$). Even if dF/dx is very small (vanishing through the $F(x)$ layers), the '+1' term ensures the gradient flowing back through the skip connection is AT LEAST $dL/d(\text{output})$ — i.e., the gradient signal can flow UNCHANGED through the identity path, providing a 'gradient highway' directly to earlier layers, regardless of how deep the network is or how poorly-behaved the $F(x)$ path's gradients are.
[Advanced] Conceptually, why is it often EASIER for a network to learn a 'residual' $F(x)$ (the difference from identity) than to learn the full desired transformation $H(x)$ directly?	If the optimal transformation for a block is CLOSE TO THE IDENTITY ($\text{output} \approx \text{input}$ — common in very deep networks where most layers make only small refinements), then: learning $F(x)=H(x)-x \approx 0$ (i.e., the residual is near zero) is a SIMPLE target — weights can stay near zero/small, easy for gradient descent to find. Learning $H(x) \approx x$ DIRECTLY (without the residual framing) requires the layer's weights to effectively learn an IDENTITY MAPPING through non-linear activations and multiple transformations — a surprisingly HARD function for a generic neural network layer to learn precisely via gradient descent (it's a very specific, non-generic target). The residual reformulation turns a 'hard to learn identity' problem into an 'easy to learn near-zero' problem.

Topic 22: Transfer Learning with CNNs

□ Concept Overview

Transfer learning reuses a CNN pretrained on a large dataset (e.g., ImageNet, 1.2M images) for a NEW, smaller dataset/task. Approaches: (1) Feature extraction — freeze pretrained layers, train only a new classifier head on top. (2) Fine-tuning — unfreeze some/all pretrained layers and

continue training (usually with a small learning rate) on the new data. Works because early/middle CNN layers learn GENERAL visual features (edges, textures, shapes) that transfer across visual tasks.

Question	Model Answer
Why does transfer learning work for CNNs — what makes pretrained features transferable across different tasks?	Early and middle CNN layers learn GENERAL-PURPOSE visual features — edges, color gradients, textures, simple shapes — these patterns are USEFUL FOR VIRTUALLY ANY VISUAL TASK, not specific to the original training task (e.g., ImageNet classification). Only the LATER layers (closer to the output) learn task-specific high-level concepts (e.g., 'is this a Golden Retriever vs a Labrador'). This means the GENERAL FEATURES learned from a large dataset (ImageNet) provide a strong starting point ('warm start') even for a completely different task (e.g., medical image classification) with much less training data than would be needed to learn these general features from scratch.
When would you choose feature extraction (frozen pretrained layers) vs fine-tuning (unfrozen layers)?	Feature extraction (freeze all pretrained layers, train only new classifier head): best when the NEW dataset is VERY SMALL (risk of overfitting if unfreezing many parameters), or when the new task is VERY SIMILAR to the pretraining task (general features are likely sufficient). Fine-tuning (unfreeze some/all layers, train with small learning rate): best when the new dataset is LARGER (enough data to safely update more parameters without overfitting), or when the new task DIFFERS significantly from the pretraining domain (e.g., medical X-rays vs natural images — even 'general' features may need adjustment).
[Advanced] Why is a SMALL learning rate typically used when fine-tuning pretrained layers, often smaller than the learning rate for the new classifier head?	The pretrained weights already encode USEFUL, well-tuned representations learned from a large dataset — a LARGE learning rate could rapidly and significantly PERTURB these weights away from their useful pretrained values, potentially DESTROYING the transferred knowledge before the new classifier head (initialized randomly) has learned anything useful to guide the fine-tuning direction ('catastrophic forgetting'). A small learning rate allows the pretrained layers to make SMALL, GRADUAL adjustments to better suit the new task, while the (randomly initialized) new head can use a LARGER learning rate since it has no useful prior values to preserve — this is sometimes implemented as DIFFERENTIAL/DISCRIMINATIVE learning rates across layers.

Topic 23: Object Detection Basics (R-CNN Family, YOLO Overview)

□ Concept Overview

Object detection requires both CLASSIFYING and LOCALIZING (bounding box) multiple objects in an image. R-CNN family (R-CNN, Fast R-CNN, Faster R-CNN): two-stage — first PROPOSE candidate regions, then CLASSIFY+REFINE each region — accurate but slower. YOLO (You Only Look Once): single-stage — directly predicts bounding boxes and classes in ONE forward pass over a grid — much faster, suitable for real-time detection, historically slightly less accurate than two-stage methods (gap has narrowed in later versions).

Question	Model Answer
What is the core difference between two-stage detectors (R-CNN family) and one-stage detectors (YOLO)?	Two-stage (R-CNN family): STAGE 1 — a region proposal mechanism generates candidate bounding boxes likely to contain objects (e.g., Region Proposal Network in Faster R-CNN). STAGE 2 — each proposed region is CLASSIFIED (what object?) and its bounding box REFINED (more precise coordinates). Generally more ACCURATE but SLOWER (two separate networks/stages). One-stage (YOLO): a SINGLE network directly predicts, for a grid of positions across the image, BOTH the bounding box coordinates AND class probabilities in ONE PASS — much FASTER (real-time capable) historically at some cost to accuracy, especially for small/overlapping objects, though the gap has narrowed significantly in modern YOLO versions.
What is the evolution from R-CNN to Fast R-CNN to Faster R-CNN?	R-CNN: runs a SEPARATE CNN forward pass for EACH of ~2000 region proposals (generated by a non-neural algorithm) — extremely slow (minutes per image). Fast R-CNN: runs the CNN ONCE on the WHOLE IMAGE, then extracts/pools features for each region proposal from this SHARED feature map (RoI pooling) — much faster (one CNN pass instead of 2000). Faster R-CNN: replaces the non-neural region proposal algorithm with a learned REGION PROPOSAL NETWORK (RPN) — makes region proposal ITSELF part of the trainable network, further speeding up and improving the pipeline (near real-time).
What is Intersection over Union (IoU) and how is it used in object detection?	$\text{IoU} = (\text{area of overlap between predicted and ground-truth bounding boxes}) / (\text{area of union of the two boxes})$. Range [0,1]. IoU=1: perfect overlap. IoU=0: no overlap. Used for: 1)

EVALUATION — a predicted box is considered a 'correct detection' if IoU with the ground truth exceeds a threshold (commonly 0.5). 2) NON-MAX SUPPRESSION — when multiple predicted boxes overlap significantly (high IoU) for the same object, keep only the highest-confidence box and SUPPRESS the others (they're likely duplicates of the same detection).

Topic 24: Image Segmentation Basics (U-Net Overview)

□ Concept Overview

Segmentation classifies EVERY PIXEL (not just the whole image or a bounding box) — producing a pixel-level mask. U-Net architecture: an ENCODER (downsampling path, like a normal CNN extracting features) followed by a DECODER (upsampling path, reconstructing full resolution), with SKIP CONNECTIONS between corresponding encoder/decoder levels — combines high-level semantic information (from deep encoder layers) with fine-grained spatial detail (from shallow encoder layers passed via skip connections), critical for precise pixel-level boundaries.

Question	Model Answer
What is image segmentation and how does it differ from classification and object detection?	Classification: ONE label for the WHOLE image ('this image contains a cat'). Object detection: bounding boxes + labels for objects ('cat at coordinates [x,y,w,h]'). Segmentation: a label for EVERY PIXEL ('pixel (10,15) is part of a cat, pixel (10,16) is background') — produces a full-resolution MASK matching the input image dimensions, where each pixel is assigned to a class. Far more granular — useful for medical imaging (precise tumor boundaries), autonomous driving (exact road/lane boundaries), etc.
What is the encoder-decoder structure of U-Net, and what is the purpose of its skip connections?	Encoder (downsampling path/contracting path): a series of conv+pooling layers progressively REDUCE spatial resolution while INCREASING the number of feature channels — extracts increasingly abstract/high-level features (like a normal CNN classifier). Decoder (upsampling path/expanding path): a series of upsampling/transposed-convolution layers progressively INCREASE spatial resolution back to the original image size, producing the final pixel-wise mask. Skip connections: directly connect each ENCODER level to the CORRESPONDING DECODER level (same spatial resolution) — this passes FINE-GRAINED

	SPATIAL DETAIL (lost during downsampling) directly to the decoder, combined with the HIGH-LEVEL SEMANTIC information from the deep encoder bottleneck — without skip connections, the decoder would have to RECONSTRUCT fine spatial detail purely from the heavily-compressed bottleneck representation, producing blurry/imprecise boundaries.
[Advanced] Why is the U-Net architecture (encoder-decoder with skip connections) widely applicable beyond pure CNNs — e.g., conceptually related to ideas in other domains?	The encoder-decoder-with-skip-connections pattern represents a GENERAL SOLUTION to a recurring problem: when a network must COMPRESS information (encoder, for understanding/abstraction) AND THEN RECONSTRUCT/GENERATE detailed output (decoder), naive compression LOSES information needed for the detailed output. Skip connections provide a 'shortcut' for preserving such information. This same pattern appears in: image-to-image translation tasks (e.g., super-resolution, style transfer), some sequence-to-sequence models (early attention mechanisms were partly motivated by similar 'information bottleneck' concerns in encoder-decoder RNNs — covered in Module E), and even ResNet's skip connections address a related (though not identical) 'information/gradient preservation across depth' concern. Recognizing this 'compress-then-reconstruct-with-shortcuts' pattern helps generalize across architectures.

MODULE D — Sequence Models: RNNs and Beyond

Topic 25: Why Sequence Models — Limitations of CNNs/MLPs for Sequential Data

□ Concept Overview

MLPs/CNNs assume FIXED-SIZE inputs and have NO inherent notion of ORDER/sequence — a sentence's word order ('dog bites man' vs 'man bites dog') would be treated identically by a model that just looks at a bag of words (or even a CNN's local patterns, to a lesser degree). Sequential data (text, time series, audio) has VARIABLE LENGTH and ORDER-DEPENDENT meaning — requiring architectures that process inputs STEP-BY-STEP while maintaining a notion of 'memory' of previous steps.

Question	Model Answer
Why can't a standard MLP directly handle variable-length sequential data like sentences?	MLPs require a FIXED-SIZE input vector (the input layer has a fixed number of neurons). Sentences/sequences have VARIABLE LENGTH (a 5-word sentence vs a 50-word sentence) — there's no natural fixed-size representation without either truncating/padding (losing information or wasting computation) or some form of AGGREGATION (e.g., averaging word embeddings — but this loses ORDER information entirely, as discussed for Average Word2Vec earlier in this curriculum).
What is the key property that sequence models (RNNs) introduce to handle this, and what is the core idea of a 'hidden state'?	RNNs process the sequence STEP-BY-STEP (one element at a time), maintaining a HIDDEN STATE (a vector) that is UPDATED at each step based on the current input AND the previous hidden state: $h_t = f(x_t, h_{t-1})$. This hidden state acts as a 'memory' or 'summary' of everything seen SO FAR in the sequence — by the end of the sequence, h_T (in principle) contains information about the ENTIRE sequence, in a FIXED-SIZE vector, REGARDLESS of the sequence's length. The SAME function f (same weights) is applied at every time step — parameter sharing across time, analogous to parameter sharing across space in CNNs.

Topic 26: Recurrent Neural Networks (RNN) — Architecture & Hidden State

□ Concept Overview

An RNN processes a sequence $x_1, \dots, x_T$ one element at a time: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$, and optionally produces output $y_t = W_{hy}h_t$ at each step. The SAME weight matrices (W_{hh} , W_{xh} , W_{hy}) are used at EVERY time step — this is the key recurrent structure. The final hidden state h_T (or all h_t 's) can be used for sequence classification, sequence generation, etc.

Question	Model Answer
Write out the basic RNN recurrence relation and explain each term.	$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$. h_t : hidden state at time t (the 'memory'). h_{t-1} : hidden state from the PREVIOUS time step. x_t : input at the current time step. W_{hh} : weight matrix applied to the previous hidden state ('how much does past memory influence the new state'). W_{xh} : weight matrix applied to the current input ('how much does the current input influence the new state'). $\tanh$: non-linearity (squashes to $(-1, 1)$, historically standard for RNNs). The SAME W_{hh} , W_{xh} , b_h are used at EVERY time step — this weight SHARING across time is the defining feature of RNNs.
What does it mean for an RNN to have 'parameter sharing across time steps', and why is this important?	The IDENTICAL weight matrices (W_{hh} , W_{xh}) are applied at EVERY time step, regardless of the sequence's length or the position within the sequence. Importance: 1) Allows the RNN to handle VARIABLE-LENGTH sequences with a FIXED number of parameters (parameters don't scale with sequence length). 2) Provides a useful INDUCTIVE BIAS — a pattern (e.g., 'negation word followed by adjective flips sentiment') learned at one position in training applies AUTOMATICALLY to the same pattern at ANY position in any sequence, similar to how CNN filters detect the same pattern anywhere in an image.
What are the different RNN architectures based on input/output sequence length relationships (one-to-many, many-to-one, many-to-many)?	One-to-many: single input, sequence output (e.g., image captioning — one image $\rightarrow$ sequence of words). Many-to-one: sequence input, single output (e.g., sentiment classification — sequence of words $\rightarrow$ single positive/negative label, using the FINAL hidden state h_T). Many-to-many (aligned): sequence input, sequence output of SAME length, one output per input step (e.g., part-of-speech tagging — each word gets a tag). Many-to-many (unaligned/Seq2Seq): sequence

input, sequence output of DIFFERENT length (e.g., translation — input and output sentences have different lengths) — requires an encoder-decoder structure (covered later).

Topic 27: Backpropagation Through Time (BPTT)

□ Concept Overview

BPTT trains RNNs by 'unrolling' the recurrence across all time steps into an equivalent feedforward computational graph, then applying standard backpropagation — but gradients for the SHARED weights (W_{hh} , W_{xh}) are SUMMED across all time steps (since the same weight is used at every step, its total gradient is the sum of its contributions at each step). For long sequences, this 'unrolled' graph is very deep — directly causing the vanishing/exploding gradient problems discussed earlier, but COMPOUNDED across TIME instead of just LAYERS.

Question	Model Answer
What does it mean to 'unroll' an RNN for backpropagation, and how does this relate to gradient computation for shared weights?	Unrolling: draw the RNN's computation EXPLICITLY for EACH time step as a separate 'copy' of the recurrence, connected in sequence — h_1 depends on x_1, h_0 ; h_2 depends on x_2, h_1 ; etc., up to h_T . This turns the recurrent computation into an equivalent (very deep) FEEDFORWARD graph, where standard backpropagation can be applied. Because the SAME weight matrices (W_{hh} , W_{xh}) appear at EVERY unrolled time step, the GRADIENT for these shared weights is the SUM of the gradient contributions from EVERY time step — $dL/dW_{hh} = \text{sum over } t \text{ of } (dL/dW_{hh} \text{ at step } t)$.
Why are vanishing/exploding gradients an ESPECIALLY severe problem for RNNs compared to standard deep feedforward networks?	In BPTT, the gradient must flow back through EVERY TIME STEP — for a sequence of length $T=100$, this is equivalent to backpropagating through a 100-LAYER network, where EVERY 'layer' uses the SAME weight matrix W_{hh} . The gradient involves a term like $(W_{hh})^T$ raised to a power related to the number of time steps — if the largest eigenvalue of W_{hh} is < 1 , this term shrinks EXPONENTIALLY (vanishing) with sequence length; if > 1 , it grows exponentially (exploding). This means VANILLA RNNs struggle to learn LONG-RANGE DEPENDENCIES (relationships between elements far apart in the sequence) — the gradient signal connecting a late time step's error to an early time step's parameters vanishes before it can have any effect, EVEN MORE severely than in deep

	feedforward nets, because the SAME matrix is repeatedly applied (compounding the effect).
What is Truncated BPTT and why is it used in practice?	For VERY LONG sequences (e.g., thousands of time steps), full BPTT (unrolling the ENTIRE sequence) is computationally/memory PROHIBITIVE — storing all intermediate activations for backprop across thousands of steps requires enormous memory. Truncated BPTT: process the sequence in CHUNKS (e.g., 50 time steps at a time), only backpropagating gradients WITHIN each chunk — the hidden state is CARRIED OVER between chunks (forward pass continues normally) but gradients are NOT backpropagated across chunk boundaries. This bounds memory usage and computation per update, at the cost of the model being unable to learn dependencies LONGER than the truncation length via gradients (though the carried-over hidden state can still propagate SOME information across chunks).

Topic 28: Vanishing/Exploding Gradients in RNNs (Long-Range Dependencies)

□ Concept Overview

Vanilla RNNs struggle to retain information across long sequences — by the time the network reaches time step 50, information from time step 1 has typically been 'overwritten' by repeated multiplication through W_{hh} (vanishing), making it nearly impossible to learn dependencies spanning long distances (e.g., subject-verb agreement across a long clause). This motivated the design of GATED architectures (LSTM, GRU) with explicit mechanisms to PRESERVE information over long ranges.

Question	Model Answer
Give a concrete NLP example illustrating why vanilla RNNs struggle with long-range dependencies.	Consider: 'The cat, which had been sitting on the windowsill for hours watching birds outside in the garden, WAS hungry.' To correctly predict 'WAS' (singular, agreeing with 'cat') rather than 'WERE', the model must REMEMBER that the subject was 'cat' (singular) across a LONG intervening clause (15+ words). A vanilla RNN's hidden state, repeatedly transformed by W_{hh} at each of these 15+ steps, tends to LOSE this early 'cat=singular' information by the time it reaches 'was/were' — the vanishing gradient means the TRAINING signal connecting this long-range dependency to

	the early 'cat' information is too weak to learn it reliably.
What design principle do LSTMs/GRUs introduce to address this, at a high level (before going into gate details)?	LSTMs/GRUs introduce an explicit MEMORY CELL/STATE with GATING MECHANISMS that allow information to be CARRIED ACROSS TIME STEPS WITH MINIMAL TRANSFORMATION when needed — analogous to ResNet's skip connections providing a 'gradient highway'. Specifically, the cell state update includes an ADDITIVE (not purely multiplicative) component, controlled by learned 'gates' that decide what to KEEP, FORGET, and ADD at each step — this additive pathway allows gradients to flow back across many time steps WITHOUT being repeatedly multiplied by the SAME matrix (the core cause of vanishing gradients in vanilla RNNs).

Topic 29: LSTM — Gates (Forget, Input, Output) & Cell State

□ Concept Overview

LSTM (Long Short-Term Memory) maintains TWO states: cell state C_t (long-term memory) and hidden state h_t (output/short-term). Three gates (each a sigmoid, output in $[0,1]$, acting as a 'how much to let through' control): FORGET gate (decides what to discard from C_{t-1}), INPUT gate (decides what new information to add to C_t), OUTPUT gate (decides what part of C_t to expose as h_t). The cell state update $C_t = \text{forget_gate} * C_{t-1} + \text{input_gate} * \text{candidate}$ is largely ADDITIVE, enabling long-range gradient flow.

Question	Model Answer
What are the three gates in an LSTM and what does each control?	Forget gate ($f_t = \text{sigmoid}(\dots)$): decides what fraction of the PREVIOUS cell state C_{t-1} to RETAIN vs DISCARD — values near 0 mean 'forget this information', near 1 mean 'keep it'. Input gate ($i_t = \text{sigmoid}(\dots)$): decides how much of the NEW CANDIDATE information (computed from current input and previous hidden state) to ADD to the cell state. Output gate ($o_t = \text{sigmoid}(\dots)$): decides what portion of the (updated) cell state C_t to EXPOSE as the hidden state h_t (which is used for predictions and passed to the next time step). All three gates are learned (have their own weight matrices), each producing values in $[0,1]$ via sigmoid, acting as 'soft switches'.

Write out the cell state update equation and explain why it helps with vanishing gradients.	$C_t = f_t * C_{t-1} + i_t * \text{candidate}_t$, where $\text{candidate}_t = \tanh(\dots)$ is the new information proposed at this step. This update is largely ADDITIVE: if $f_t \approx 1$ (forget gate near 1, 'remember everything') and $i_t \approx 0$ (input gate near 0, 'add nothing new'), then $C_t \approx C_{t-1}$ — the cell state passes through ESSENTIALLY UNCHANGED. During backpropagation, $dC_t/dC_{t-1} \approx f_t$ (close to 1 in this case) — NOT involving repeated multiplication by a weight matrix raised to high powers (the cause of vanishing gradients in vanilla RNNs). This allows gradients (and information) to flow across MANY time steps with much less decay, as long as the forget gate learns to stay close to 1 for information that needs to be preserved long-term.
What is the difference between the cell state C_t and the hidden state h_t in an LSTM?	Cell state C_t : the 'long-term memory' — updated mostly ADDITIVELY (via forget/input gates), designed to PRESERVE information across many time steps with minimal decay. Hidden state h_t : the 'working memory'/output — computed as $h_t = o_t * \tanh(C_t)$, a GATED, NON-LINEAR transformation of the cell state, used for the CURRENT prediction and passed as input to the next time step's gate computations. Conceptually: C_t is the 'long-term storage', h_t is 'what's currently being actively used/exposed' — a FILTERED view of the long-term storage relevant to the current step.

Topic 30: GRU — Simplified Gating

□ Concept Overview

GRU (Gated Recurrent Unit) simplifies LSTM: combines the forget and input gates into a single UPDATE gate, and merges the cell state and hidden state into ONE state vector. Two gates total: UPDATE gate (how much of the previous state to keep vs replace with new candidate) and RESET gate (how much of the previous state to use when computing the new candidate). Fewer parameters than LSTM, often comparable performance, faster to train — a common practical choice.

Question	Model Answer
How does GRU simplify the LSTM architecture, and what are its two gates?	GRU MERGES the LSTM's cell state and hidden state into a SINGLE state vector h_t , and combines the forget+input gates into ONE UPDATE GATE. Update gate (z_t): controls the INTERPOLATION between the previous hidden

	state h_{t-1} and a new candidate state — $h_t = (1-z_t)*h_{t-1} + z_t*candidate_t$ (analogous to LSTM's forget+input combined into one 'mixing' parameter). Reset gate (r_t): controls how much of the PREVIOUS hidden state is used when COMPUTING the new candidate — allows the model to 'reset'/ignore past state when starting a new, unrelated segment of information.
What are the practical tradeoffs between using GRU vs LSTM?	GRU has FEWER PARAMETERS (2 gates + merged state vs LSTM's 3 gates + 2 state vectors) — FASTER to train and run, lower memory, can be advantageous for SMALLER datasets (less overfitting risk) or when computational efficiency matters. LSTM has MORE EXPRESSIVE CAPACITY (separate cell/hidden states, finer-grained gating) — may perform slightly BETTER on tasks requiring very long-range dependencies or LARGE datasets where the extra parameters can be effectively utilized. In practice: performance differences are often SMALL and TASK-DEPENDENT — many practitioners try both and pick based on empirical validation performance, with GRU often a reasonable 'faster first try'.

Topic 31: Bidirectional RNNs

□ Concept Overview

A standard ('unidirectional') RNN processes a sequence left-to-right — at time t , it only has information from positions $1...t$ (the PAST). For many tasks (e.g., 'fill in the blank' or named entity recognition), FUTURE context (positions $t+1...T$) is also informative. A Bidirectional RNN runs TWO separate RNNs: one left-to-right (forward) and one right-to-left (backward), and CONCATENATES their hidden states at each position — giving each position access to BOTH past and future context.

Question	Model Answer
What limitation of unidirectional RNNs do Bidirectional RNNs address, and how?	Unidirectional RNN at time t: hidden state h_t only depends on $x_1, ..., x_t$ (information from the PAST/left context only) — it has NO information about what comes AFTER position t. For many tasks, FUTURE context matters: e.g., disambiguating 'bank' in 'I went to the bank to ___' requires seeing what follows ('deposit money' vs 'watch the sunset over the river'). Bidirectional RNN: runs TWO independent RNNs — a FORWARD RNN (processes left-to-right,

Why can't a Bidirectional RNN be used for AUTOREGRESSIVE generation tasks (e.g., language modeling, generating text left-to-right)?	producing $h_{t_forward}$) and a BACKWARD RNN (processes right-to-left, producing $h_{t_backward}$) — then CONCATENATES [$h_{t_forward}$, $h_{t_backward}$] at each position, giving each position's representation access to information from BOTH directions. Autoregressive generation predicts the NEXT token based ONLY on PREVIOUS tokens (the future tokens DON'T EXIST YET at generation time — they're what we're trying to predict). A Bidirectional RNN's backward pass REQUIRES the FULL SEQUENCE (including future tokens) to compute $h_{t_backward}$ — this information SIMPLY ISN'T AVAILABLE during generation (you can't 'look at the future word' to help generate the current word, since that future word hasn't been generated yet). Bidirectional architectures are appropriate for tasks where the FULL INPUT SEQUENCE IS AVAILABLE UPFRONT (e.g., classification, tagging, encoding for translation) — NOT for autoregressive generation, which is fundamentally restricted to using only past context (this distinction becomes important again for BERT vs GPT in Module G).
--	---

Topic 32: Sequence-to-Sequence (Seq2Seq) Models

□ Concept Overview

Seq2Seq handles tasks where INPUT and OUTPUT sequences may have DIFFERENT lengths (e.g., translation: a 5-word English sentence might translate to a 7-word French sentence). Structure: an ENCODER RNN processes the entire input sequence into a FIXED-SIZE 'context vector' (typically the encoder's final hidden state); a DECODER RNN then generates the output sequence ONE TOKEN AT A TIME, conditioned on this context vector (and previously generated tokens).

Question	Model Answer
What problem does the Seq2Seq architecture solve that simple 'aligned' many-to-many RNNs cannot?	Aligned many-to-many RNNs (e.g., POS tagging) produce ONE output PER INPUT TIME STEP — input and output sequences must have the SAME LENGTH, with a clear one-to-one POSITIONAL correspondence. Many real tasks DON'T have this property: translation (different languages have different word counts/orderings for the 'same' sentence), summarization (output is much SHORTER than input), question answering (output length is unrelated to input length).

	Seq2Seq DECOUPLES input and output sequence lengths/structure entirely — the encoder compresses the WHOLE input into a context representation, and the decoder GENERATES an output sequence of WHATEVER LENGTH IS APPROPRIATE, independent of the input's length.
Describe the basic encoder-decoder Seq2Seq architecture and its key bottleneck.	Encoder: an RNN (often LSTM/GRU) processes the input sequence token-by-token, producing a final hidden state — the 'context vector' — intended to summarize the ENTIRE input sequence's meaning in a SINGLE FIXED-SIZE VECTOR. Decoder: another RNN, INITIALIZED with this context vector as its starting hidden state, generates the output sequence ONE TOKEN AT A TIME — at each step, it takes the PREVIOUSLY GENERATED token (or a special start token initially) as input, and produces the next token, until a special 'end' token is generated. KEY BOTTLENECK: the ENTIRE input sequence's information must be SQUEEZED into ONE FIXED-SIZE VECTOR (the context vector) — for LONG input sequences, this is an information bottleneck (analogous to the autoencoder bottleneck discussed for PCA) — this exact limitation is what MOTIVATES the ATTENTION MECHANISM (Module E).

Topic 33: Encoder-Decoder Architecture — Deeper Dive

□ Concept Overview

Building on Seq2Seq: the encoder-decoder paradigm generalizes beyond RNNs — it's an architectural PATTERN (encode input into a representation, decode representation into output) that recurs throughout deep learning: Seq2Seq RNNs, U-Net (Module C), and later, the Transformer (Module F) all follow encoder-decoder structures, though the INTERNAL MECHANISM (RNN recurrence, conv+pooling, self-attention) differs.

Question	Model Answer
What is the encoder-decoder pattern as a GENERAL architectural principle, beyond just RNN Seq2Seq?	Encoder-decoder is a TWO-STAGE pattern: an ENCODER transforms the input into some INTERMEDIATE REPRESENTATION (a 'summary' or 'compressed code' capturing the input's relevant information), and a DECODER transforms this intermediate representation into the desired OUTPUT. The pattern is AGNOSTIC

	to the specific mechanism used: RNN Seq2Seq (encoder/decoder are RNNs, intermediate representation is a context vector), U-Net (encoder/decoder are CNN downsampling/upsampling paths, intermediate representation is the bottleneck feature map + skip connections), Transformer (encoder/decoder are stacks of self-attention layers, intermediate representation is a SEQUENCE of context vectors, one per input position — Module F). Recognizing this PATTERN helps understand new architectures by asking 'what is the encoder, what is the decoder, and what is the intermediate representation here?'
What is 'teacher forcing' during Seq2Seq training, and why is it used?	During TRAINING, when the decoder generates output token $y_{\hat{t}}$ at step t, the NEXT step's input would normally be $y_{\hat{t}}$ (the model's OWN prediction). Teacher forcing: instead, FEED THE TRUE/GROUND-TRUTH token y_t (from the training data) as input to the next step, REGARDLESS of what the model predicted at step t. Why: 1) STABILIZES/SPEEDS UP training — early in training, the model's predictions are mostly wrong; feeding these wrong predictions back in would compound errors and provide a very noisy training signal. 2) Allows PARALLELIZATION — since the decoder's inputs at all time steps are KNOWN IN ADVANCE (the ground truth sequence), all time steps' losses can in principle be computed without sequential dependency on the model's own (possibly slow/sequential) predictions. DOWNSIDE: creates a train/inference MISMATCH ('exposure bias') — at INFERENCE, the model MUST use its OWN (possibly imperfect) predictions as subsequent inputs, since ground truth isn't available — a discrepancy the model never experienced during training.

MODULE E — Attention Mechanism: The Bridge to Transformers

Topic 34: Limitations of Seq2Seq — The Fixed Context Vector Bottleneck

□ Concept Overview

In basic Seq2Seq, the ENTIRE input sequence (regardless of length — 5 words or 50 words) must be compressed into ONE fixed-size context vector (the encoder's final hidden state). For long sequences, this is an information bottleneck — early-sequence information tends to be 'diluted'/overwritten by the time the encoder reaches the end, and the decoder has NO way to 'look back' at specific parts of the input when generating different output tokens. Performance degrades sharply for longer sequences.

Question	Model Answer
Why does basic Seq2Seq performance degrade significantly for LONGER input sequences?	The encoder's final hidden state (context vector) is a SINGLE FIXED-SIZE VECTOR that must somehow encode the ENTIRE input sequence's meaning. For a 5-word sentence, this is plausible. For a 50-word sentence (or a long document), squeezing ALL the relevant information into the SAME fixed-size vector becomes increasingly LOSSY — much like trying to summarize an entire book in one sentence vs one paragraph. Additionally, due to the RNN's recurrent nature, information from EARLY in the sequence has been transformed/diluted through MANY recurrent steps by the time the encoder finishes — making it especially hard to retain early information for long sequences.
Give a concrete translation example illustrating WHY the decoder needs to 'look back' at DIFFERENT parts of the input for DIFFERENT output words.	Translating 'The red house' to French ('La maison rouge') requires REORDERING — French places the adjective AFTER the noun. When generating 'maison' (house), the decoder should focus on the INPUT WORD 'house'. When generating 'rouge' (red), it should focus on 'red'. A SINGLE static context vector cannot represent 'right now, focus on word 3 of the input; next, focus on word 2' — it's ONE FIXED SUMMARY used for the ENTIRE decoding process. What's needed: a mechanism for the decoder to DYNAMICALLY select/weight DIFFERENT parts of the input representation AT EACH DECODING STEP — this is exactly what ATTENTION provides.

Topic 35: Attention Mechanism — Intuition (Bahdanau/Luong Attention)

□ Concept Overview

Instead of using ONLY the encoder's FINAL hidden state, attention gives the decoder access to ALL encoder hidden states (one per input position). At each decoding step, attention computes a WEIGHTED AVERAGE of all encoder hidden states, where the weights are DYNAMICALLY COMPUTED based on how RELEVANT each input position is to the CURRENT decoding step. This 'soft alignment' lets the decoder focus on different input positions for different output positions — directly addressing the fixed-context-vector bottleneck.

Question	Model Answer
What is the core idea of attention, described WITHOUT formulas?	Instead of compressing the ENTIRE input into one fixed vector that's used for ALL decoding steps, attention keeps ALL of the encoder's hidden states (one per input position) AVAILABLE. At EACH decoding step, the decoder computes a SET OF RELEVANCE/SIMILARITY SCORES between its current state and EACH encoder hidden state, converts these scores to WEIGHTS (via softmax, summing to 1), and takes a WEIGHTED AVERAGE of the encoder hidden states using these weights — producing a 'context vector' that's CUSTOM-TAILORED to the CURRENT decoding step, dynamically emphasizing the most relevant input positions.
What is the difference between Bahdanau (additive) and Luong (multiplicative) attention at a high level?	Both compute relevance scores between the decoder's current state and each encoder hidden state, then softmax+weighted-average — the difference is in HOW the score is computed. Bahdanau attention: uses a SMALL FEEDFORWARD NEURAL NETWORK (with learnable weights) to compute the score from the concatenation of decoder state and encoder hidden state — more flexible/expressive, slightly more computation. Luong attention: uses a simpler DOT PRODUCT (or a single learned weight matrix applied before the dot product) between decoder state and encoder hidden state — computationally cheaper, and this DOT-PRODUCT-BASED scoring is the DIRECT PRECURSOR to the scaled dot-product attention used in Transformers (Module F).
How does attention solve the long-sequence degradation problem of basic Seq2Seq?	Because attention gives the decoder DIRECT ACCESS to ALL encoder hidden states (not just a single compressed summary), the 'distance' information needs to travel is SHORTER —

relevant information from ANY input position can be DIRECTLY ATTENDED TO at the moment it's needed, rather than having to survive being compressed into (and later extracted from) a single bottleneck vector. Empirically, attention-augmented Seq2Seq models show much LESS performance degradation as sequence length increases compared to basic Seq2Seq — the attention weights can be VISUALIZED as an 'alignment matrix' showing which input words the model attends to when generating each output word, often matching intuitive human alignments (e.g., correctly aligning 'rouge' with 'red').

Topic 36: Attention as Weighted Average — Query, Key, Value Framing

□ Concept Overview

The modern formalization of attention uses three vectors per input element: QUERY (Q) — 'what am I looking for?' (from the decoder/current position), KEY (K) — 'what do I contain?' (from each encoder/input position, used to match against queries), VALUE (V) — 'what information do I provide?' (from each encoder/input position, used in the weighted average). Attention score = similarity(Query, Key) for each position; weights = softmax(scores); output = weighted sum of Values using these weights.

Question	Model Answer
Explain the Query, Key, Value (Q, K, V) framing of attention using an analogy.	Analogy: a LIBRARY SEARCH. QUERY = your search question ('books about deep learning'). KEY = the catalog TAGS/INDEX TERMS of each book ('this book is tagged: neural networks, AI, math'). VALUE = the ACTUAL CONTENT of each book (what you'd actually read/use). The search MATCHES your query against each book's KEY (tags) to compute a RELEVANCE SCORE — books with tags closely matching 'deep learning' get HIGH scores. These scores become WEIGHTS, and you receive a WEIGHTED COMBINATION of the books' VALUES (content) — heavily weighted toward the most relevant books, but with SOME contribution from all books (soft attention, not hard selection).
Write out the general attention formula and explain each component.	$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$ $Q \cdot K^T$: computes the DOT PRODUCT (similarity) between the query and EVERY key —

	a higher dot product means the query and that key are more 'aligned'/relevant. $1/\sqrt{d_k}$: SCALING factor (d_k = dimension of the key vectors) — prevents the dot products from becoming too large in magnitude (which would make softmax outputs extremely 'peaked'/saturated, causing vanishing gradients through softmax — this scaling is specifically important in Transformers, covered in Module F). softmax(...): converts the scaled scores into WEIGHTS that sum to 1 (a probability distribution over input positions). * V: computes the WEIGHTED SUM of the value vectors using these attention weights — the final output, a context vector CUSTOMIZED for this specific query.
In the original Seq2Seq+attention setup, what serve as the Query, Keys, and Values?	Query: the DECODER's current hidden state (representing 'what information do I need right now to generate the next output token?'). Keys AND Values: BOTH come from the ENCODER's hidden states at each input position (in the original formulation, Keys and Values are often the SAME as the encoder hidden states — this distinction between Keys and Values as SEPARATE learned projections becomes more pronounced/explicit in Transformer self-attention, Module F). The decoder's query is compared against each encoder position's key to determine relevance, and the weighted combination of encoder values (hidden states) provides the dynamically-computed context for this decoding step.

Topic 37: Self-Attention — Relating All Positions to Each Other

□ Concept Overview

Self-attention applies the Q/K/V mechanism WITHIN a SINGLE sequence — every position attends to EVERY OTHER position in the SAME sequence (including itself), computing Q, K, V all from the SAME input sequence (via different learned projection matrices W_Q , W_K , W_V). This allows the model to directly capture relationships between ANY two positions in a sequence, REGARDLESS of their distance — unlike RNNs where information must flow sequentially through intermediate positions.

Question

Model Answer

What is self-attention and how does it differ from the encoder-decoder attention described in Topic 35-36?	Encoder-decoder attention (Topics 35-36): Query comes from the DECODER, Keys/Values come from the ENCODER — attention RELATES TWO DIFFERENT SEQUENCES (input sequence and output-so-far). Self-attention: Query, Key, AND Value ALL come from the SAME sequence — each position in a sequence computes its Q, K, V (via learned projections W_Q, W_K, W_V applied to that position's representation), and ATTENDS TO ALL POSITIONS IN THE SAME SEQUENCE (including itself) — capturing relationships WITHIN a single sequence (e.g., 'this pronoun refers to that noun earlier in the sentence').
Give a concrete NLP example of what self-attention can capture that's hard for RNNs.	Sentence: 'The animal didn't cross the street because IT was too tired.' Resolving 'IT' requires connecting it to 'animal' (not 'street') — a relationship between word 1 and word 10, with 8 words in between. In an RNN, this information must flow SEQUENTIALLY through all 8 intermediate hidden states (subject to vanishing gradients over this distance, as discussed in Module D). In self-attention, word 'IT' can DIRECTLY compute a high attention score with 'animal' in a SINGLE STEP — REGARDLESS of the 8-word distance — the path length between ANY two positions is CONSTANT (= 1 attention computation), not proportional to their distance, as in RNNs.
[Advanced] What computational/parallelization advantage does self-attention have over RNNs, and what is its computational COST tradeoff?	PARALLELIZATION advantage: in an RNN, h_t depends on h_{t-1} — computation is INHERENTLY SEQUENTIAL (can't compute h_5 before h_4). In self-attention, the attention scores for ALL pairs of positions can be computed SIMULTANEOUSLY (it's just a matrix multiplication $Q \cdot K^T$ over the whole sequence at once) — highly PARALLELIZABLE on GPUs/TPUs, dramatically speeding up TRAINING for long sequences compared to RNNs. COST tradeoff: self-attention's $Q \cdot K^T$ computation is $O(n^2 \cdot d)$ where n=sequence length — QUADRATIC in sequence length (every position attends to every other position), compared to RNN's $O(n \cdot d)$ (LINEAR in sequence length, but sequential). For VERY long sequences (thousands of tokens), this quadratic cost becomes a significant computational bottleneck — motivating research into efficient/sparse attention variants for long-context models.

MODULE F — Transformers

Topic 38: 'Attention is All You Need' — Motivation: Replacing Recurrence

□ Concept Overview

The 2017 'Attention is All You Need' paper proposed the TRANSFORMER — an architecture built ENTIRELY from self-attention and feedforward layers, with NO recurrence (no RNN/LSTM) at all. Motivation: RNNs' sequential nature prevents parallelization (slow training) AND suffers from long-range dependency issues (even with LSTM gating). Self-attention provides DIRECT connections between any two positions (solving long-range dependencies) AND is fully parallelizable (solving the training speed bottleneck) — Transformers became the foundation for virtually all modern large language models.

Question	Model Answer
What were the two main problems with RNN/LSTM-based sequence models that the Transformer architecture was designed to solve?	1) SEQUENTIAL COMPUTATION: RNNs process tokens one at a time (h_t depends on h_{t-1}) — cannot be parallelized across the sequence dimension, making training on long sequences/large datasets SLOW, especially on GPUs designed for massively parallel computation. 2) LONG-RANGE DEPENDENCIES: even with LSTM/GRU gating, information must still pass THROUGH intermediate time steps to connect distant positions — for VERY long sequences, this remains imperfect. Self-attention solves BOTH: (1) all pairwise attention scores computed in parallel via matrix multiplication, (2) ANY two positions connect via a SINGLE attention computation regardless of distance.
At a high level, what are the main components of the Transformer architecture?	The Transformer follows an ENCODER-DECODER structure (similar in SPIRIT to Seq2Seq, Module D), but BOTH encoder and decoder are built from STACKS of identical layers, each containing: MULTI-HEAD SELF-ATTENTION (allows each position to attend to all other positions), POSITION-WISE FEEDFORWARD NETWORKS (applied independently to each position), RESIDUAL CONNECTIONS + LAYER NORMALIZATION (around each sub-layer, for training stability, as discussed in Modules B/C), and POSITIONAL ENCODINGS (added to input embeddings, since self-attention has NO inherent notion of

position/order — unlike RNNs where order is implicit in the recurrence).

Topic 39: Transformer Architecture Overview (Encoder-Decoder)

□ Concept Overview

The original Transformer has an ENCODER STACK (6 identical layers, each with self-attention + feedforward) that processes the input sequence into a sequence of context-aware representations, and a DECODER STACK (6 identical layers, each with MASKED self-attention + encoder-decoder cross-attention + feedforward) that generates the output sequence autoregressively, attending to both its own previous outputs AND the encoder's representations.

Question	Model Answer
What are the three types of attention used in the Transformer's decoder, and what does each attend to?	1) Masked self-attention: decoder positions attend to OTHER DECODER positions, but ONLY to positions BEFORE the current one (masked/hidden future positions) — preserves the autoregressive property (can't 'cheat' by looking at future tokens it's supposed to predict). 2) Encoder-decoder cross-attention: decoder positions (as QUERIES) attend to ENCODER output positions (as KEYS/VALUES) — this is the DIRECT analog of the Bahdanau/Luong attention from Module E, letting the decoder 'look at' relevant parts of the input. 3) (Each layer also has) position-wise feedforward: applied independently to each position after the attention sub-layers.
Why does the encoder use UNMASKED (full/bidirectional) self-attention while the decoder uses MASKED self-attention?	The ENCODER processes the COMPLETE INPUT SEQUENCE, which is FULLY AVAILABLE upfront (e.g., the entire source sentence for translation) — there's no reason to restrict attention; each position can benefit from FULL BIDIRECTIONAL CONTEXT (both earlier and later positions in the input). The DECODER GENERATES output AUTOREGRESSIVELY (one token at a time, left-to-right) — at the moment of generating token t , tokens $t+1$, $t+2$, ... DON'T EXIST YET (that's what we're predicting). MASKING enforces this: position t 's self-attention can only attend to positions $\leq t$, ensuring TRAINING (with teacher forcing, seeing the full target sequence) doesn't 'cheat' by looking at future ground-truth tokens that wouldn't be

available during actual generation/inference.

Topic 40: Multi-Head Self-Attention — Mathematical Formulation

□ Concept Overview

Instead of computing ONE attention function, multi-head attention computes MULTIPLE attention 'heads' IN PARALLEL — each head has its OWN learned W_Q , W_K , W_V projection matrices (projecting to a smaller dimension $d_k = d_{\text{model}}/\text{num_heads}$), computes attention independently, and the outputs of all heads are CONCATENATED and linearly projected back to d_{model} . Each head can learn to attend to DIFFERENT types of relationships (e.g., one head for syntactic dependencies, another for coreference).

Question	Model Answer
Why use MULTIPLE attention heads instead of one large attention computation?	A SINGLE attention head computes ONE weighted average per position — this AVERAGING can 'blur together' MULTIPLE DIFFERENT TYPES of relevant relationships into one representation (e.g., if a word relates to one other word SYNTACTICALLY and a DIFFERENT word SEMANTICALLY, a single attention distribution must compromise between attending to both). MULTIPLE HEADS, each with SEPARATE learned projections, allow the model to attend to DIFFERENT TYPES OF RELATIONSHIPS SIMULTANEOUSLY IN PARALLEL — empirically, different heads often specialize (e.g., some heads learn syntactic dependency patterns, others learn coreference/positional patterns) — analogous to how multiple CNN FILTERS in a layer detect different patterns (Module C).
How are the dimensions split across heads, and what is the computational cost compared to single-head attention with the same total dimension?	If the model dimension is d_{model} (e.g., 512) and there are h heads (e.g., 8), each head operates on dimension $d_k = d_{\text{model}}/h = 64$. Each head computes its OWN Q, K, V projections (to dimension 64) and its OWN attention output (dimension 64); the h outputs (each 64-dim) are CONCATENATED back to $d_{\text{model}}=512$, then passed through a final linear projection. The TOTAL computational cost is approximately THE SAME as a single attention head operating on the full d_{model} dimension (since each head does proportionally LESS work on a smaller dimension) — multi-head attention gives the

[Advanced] Why is the scaling factor $1/\sqrt{d_k}$ in 'scaled dot-product attention' important, especially with multi-head attention's smaller d_k per head?	REPRESENTATIONAL benefit of multiple 'relationship subspaces' at roughly the SAME computational cost as single-head attention. The dot product $Q \cdot K$ (sum of d_k products) has VARIANCE that GROWS with d_k (assuming Q, K components have unit variance, $\text{Var}(Q \cdot K) \approx d_k$). For LARGE d_k, dot products can become LARGE in magnitude — pushing softmax inputs into regions where softmax becomes very 'PEAKED' (near one-hot), and crucially, where softmax's GRADIENT becomes very SMALL (saturated) — similar to the Sigmoid/Tanh vanishing gradient issue (Module B). Dividing by $\sqrt{d_k}$ NORMALIZES the dot product's variance back to approximately 1, REGARDLESS of d_k — keeping softmax in a well-behaved, non-saturated regime, ensuring healthy gradients flow through the attention mechanism during training. This is a small but CRITICAL detail for training stability.
--	---

Topic 41: Positional Encoding — Why and How

□ Concept Overview

Self-attention is PERMUTATION-INVARIANT — it has NO inherent notion of token ORDER/POSITION ($\text{attention}(Q, K, V)$ gives the same result regardless of how you reorder the input positions, modulo relabeling). But ORDER MATTERS in language ('dog bites man' != 'man bites dog'). Positional encodings ADD position-dependent information to each token's embedding BEFORE it enters the self-attention layers — the original Transformer used FIXED sinusoidal functions ($\sin/\cos$ of different frequencies based on position), allowing the model to infer relative and absolute positions.

Question	Model Answer
Why does self-attention NEED an explicit positional encoding, unlike RNNs?	RNNs process tokens SEQUENTIALLY — the ORDER is IMPLICIT in the recurrence itself (h_t is computed AFTER h_{t-1}, inherently encoding 'this came after that'). Self-attention computes attention scores via $Q \cdot K$ for ALL PAIRS SIMULTANEOUSLY, with NO inherent reference to WHICH position is 'first' vs 'last' — mathematically, if you PERMUTED the input tokens' order, the SET of attention scores would be the SAME (just relabeled) — the mechanism is PERMUTATION-INVARIANT/order-agnostic by default. Since word ORDER carries crucial meaning in language, positional information must

	be EXPLICITLY INJECTED — this is the role of positional encodings, added to (or combined with) token embeddings before the first attention layer.
How does the original Transformer's sinusoidal positional encoding work, at a high level?	For each position 'pos' and each embedding dimension 'i', the positional encoding value is $\sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$ for even dimensions and $\cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$ for odd dimensions — different dimensions use DIFFERENT FREQUENCIES of sine/cosine waves. This produces a UNIQUE 'fingerprint' vector for each position, and importantly, the relationship between encodings at DIFFERENT POSITIONS (e.g., position 5 vs position 10) can be expressed as a LINEAR FUNCTION (rotation) — allowing the model to potentially learn to attend based on RELATIVE positions (e.g., 'attend to the token 2 positions back') via linear operations on these encodings, not just absolute positions.
What is the difference between FIXED (sinusoidal) positional encodings and LEARNED positional embeddings, and which do modern models often use?	Fixed sinusoidal: a DETERMINISTIC FUNCTION (no learnable parameters) — same formula regardless of training data; CAN, in principle, GENERALIZE to sequence lengths LONGER than seen during training (the formula extends to any position). Learned positional embeddings: a LOOKUP TABLE of learnable vectors, one per position up to some MAXIMUM length — these are TRAINED along with the rest of the model, potentially capturing position-specific patterns more flexibly, but CANNOT handle positions BEYOND the maximum length seen during training (no embedding exists for position 1000 if trained only up to 512). Many modern models (BERT, GPT-2) use LEARNED positional embeddings (simplicity, empirically comparable performance for typical sequence lengths), though more recent approaches (e.g., RoPE - rotary position embeddings, ALiBi) have developed further refinements for better LENGTH GENERALIZATION in very large language models.

Topic 42: Feed-Forward Layers, Residual Connections & Layer Norm in Transformers

□ Concept Overview

Each Transformer layer contains: (1) a multi-head self-attention sub-layer, (2) a POSITION-WISE FEEDFORWARD NETWORK (a small 2-layer MLP, applied IDENTICALLY and INDEPENDENTLY to each position) — providing additional non-linear transformation capacity beyond attention's (linear, weighted-average) operations. BOTH sub-layers are wrapped with RESIDUAL CONNECTIONS (Module C concept, applied here) and LAYER NORMALIZATION (Module B concept) — i.e., $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$ — critical for training very deep Transformer stacks.

Question	Model Answer
What is the role of the position-wise feedforward network in each Transformer layer, given that self-attention already provides significant computation?	Self-attention is fundamentally a WEIGHTED AVERAGE (linear combination) of value vectors — even with non-linear softmax for computing weights, the OUTPUT for each position is a LINEAR combination of OTHER positions' (linearly projected) values. The position-wise feedforward network (typically: Linear -> activation (ReLU/GELU) -> Linear, applied INDEPENDENTLY to EACH position) introduces ADDITIONAL NON-LINEAR TRANSFORMATION CAPACITY at EACH position — allowing the model to perform more complex, non-linear processing of the information that attention has GATHERED/COMBINED from other positions. Attention handles 'WHICH information to combine FROM WHERE'; the feedforward network handles 'WHAT NON-LINEAR PROCESSING to do WITH that combined information'.
Why are residual connections and layer normalization essential for training deep Transformer stacks (e.g., 12, 24, or 96+ layers)?	As discussed in Modules B/C: very deep networks suffer from vanishing/exploding gradients without mitigation. RESIDUAL CONNECTIONS ($\text{output} = x + \text{Sublayer}(x)$) provide a 'gradient highway' allowing gradients to flow directly through the IDENTITY path to earlier layers, regardless of depth — ESSENTIAL for training Transformers with dozens to HUNDREDS of layers (modern LLMs). LAYER NORMALIZATION (normalizing across the FEATURE dimension for each token/position, as discussed in Module B) stabilizes the SCALE of activations flowing through this deep residual stack, preventing activations from growing unboundedly across many residual additions, and is preferred over BatchNorm because it works per-TOKEN independent of batch size and sequence length — both essential properties for variable-length sequence processing.

Topic 43: Masked Self-Attention (Decoder)

□ Concept Overview

In the decoder's self-attention, a MASK is applied to the attention scores BEFORE softmax: scores for positions AFTER the current position are set to -infinity (so after softmax, their weight becomes 0). This ensures position t 's representation depends ONLY on positions $1...t$ — preserving the AUTOREGRESSIVE property required for valid left-to-right generation, while still allowing PARALLEL TRAINING (the full target sequence is processed at once, with masking preventing 'cheating').

Question	Model Answer
How is masking implemented in masked self-attention, mechanically?	Before applying softmax to the attention scores ($Q \cdot K^T / \sqrt{d_k}$), a MASK MATRIX is ADDED to the scores: for any pair (query position i , key position j) where $j > i$ (key is in the FUTURE relative to the query), set the corresponding score to -INFINITY (or a very large negative number). After softmax, $\exp(-\text{infinity}) \approx 0$ — these 'future' positions receive ZERO attention weight. This is implemented as a simple ADDITIVE MASK (an upper-triangular matrix of -infinity values added to the score matrix) — computationally cheap, and allows the ENTIRE masked self-attention computation to still be done via PARALLEL matrix operations (NOT requiring sequential processing, unlike RNNs).
[Advanced] How does masked self-attention enable PARALLEL TRAINING of an autoregressive model, given that generation itself is sequential?	During TRAINING (with teacher forcing, Module D): the FULL target sequence (ground truth) is available. Masked self-attention computes, IN ONE PARALLEL PASS, the representation for EVERY position t — where position t 's representation correctly depends ONLY on positions $1...t$ (due to masking) — exactly matching what would be available during ACTUAL AUTOREGRESSIVE GENERATION at that position. So the model can be trained to predict ALL positions' next tokens SIMULTANEOUSLY (one parallel forward+backward pass over the whole sequence), even though the underlying TASK (generation) is sequential. INFERENCE/generation REMAINS sequential (must generate token 1, then token 2 using token 1, etc. — token $t+1$ doesn't exist until generated) — masking enables parallel TRAINING despite the sequential nature of the GENERATION task itself.

Topic 44: Transformer Training — Teacher Forcing (Revisited for Transformers)

□ Concept Overview

Building on Module D's teacher forcing concept: in Transformer training, the DECODER receives the ENTIRE target sequence (shifted right by one position) as input ALL AT ONCE, with masked self-attention ensuring position t only 'sees' positions $1 \dots t-1$ of this input — effectively implementing teacher forcing for ALL positions in PARALLEL within a single forward pass, a major efficiency advantage over RNN Seq2Seq (which required SEQUENTIAL decoder steps even WITH teacher forcing).

Question	Model Answer
How does teacher forcing in Transformers differ from teacher forcing in RNN Seq2Seq, in terms of computational efficiency?	RNN Seq2Seq WITH teacher forcing: even though the ground-truth tokens are KNOWN for all positions, the decoder RNN must STILL process them SEQUENTIALLY (h_t depends on h_{t-1}) — teacher forcing only affects WHAT input is fed at each step (ground truth vs model's own prediction), not the SEQUENTIAL nature of computation itself. Transformer WITH teacher forcing: the ENTIRE target sequence (ground truth, shifted) is fed as input SIMULTANEOUSLY, and masked self-attention computes ALL positions' outputs IN ONE PARALLEL PASS (each position's computation only 'looks at' earlier positions due to masking, but this masking is just a computational CONSTRAINT within a single parallel operation, not a SEQUENTIAL DEPENDENCY between separate computational steps). This is a MAJOR speedup for TRAINING — Transformers can process an entire sequence's training loss in roughly the time of ONE forward pass, vs RNNs requiring T sequential steps for a length- T sequence.
[Advanced] Despite this training-time parallelization, why is INFERENCE (generation) with a Transformer decoder STILL sequential/slow, and what does this imply about Transformer LLM deployment costs?	At INFERENCE, the ground-truth target sequence DOESN'T EXIST (that's what we're generating) — so the 'feed the whole target sequence at once' trick is unavailable. Generation MUST proceed token-by-token: generate token 1, APPEND it to the input, re-run the (now longer) sequence through the decoder to generate token 2, append, re-run for token 3, etc. — T SEQUENTIAL FORWARD PASSES for a length- T output (each pass also getting progressively MORE EXPENSIVE as the sequence grows, due to attention's $O(n^2)$ cost). This is why TRAINING large language models can be heavily parallelized (cheaper per-token, relatively), but INFERENCE/generation remains

comparatively expensive and LATENCY-SENSITIVE — driving substantial engineering effort (KV-caching to avoid recomputation, speculative decoding, quantization) specifically to speed up the SEQUENTIAL generation process in deployed LLMs.

MODULE G — Transformer-Based Models & Modern NLP

Topic 45: BERT — Masked Language Modeling & Bidirectional Context

□ Concept Overview

BERT (Bidirectional Encoder Representations from Transformers) uses ONLY the Transformer ENCODER (no decoder) — every token can attend to EVERY OTHER token (no masking), giving FULL bidirectional context. Pretrained via MASKED LANGUAGE MODELING (MLM): randomly mask ~15% of input tokens, train the model to PREDICT the masked tokens using BOTH left AND right context. Also trained with Next Sentence Prediction (NSP). After pretraining, FINE-TUNED on downstream tasks (classification, NER, QA) by adding a small task-specific head.

Question	Model Answer
What is Masked Language Modeling (MLM) and why does it require an encoder-only (bidirectional) architecture?	MLM: randomly REPLACE ~15% of input tokens with a special [MASK] token; train the model to PREDICT the ORIGINAL token at each masked position, using the SURROUNDING context (both BEFORE and AFTER the masked position). This REQUIRES bidirectional context — predicting a masked word benefits from seeing words on BOTH sides ('The [MASK] sat on the mat' — 'cat' is suggested by both 'The___' and '___sat on the mat'). A DECODER's masked self-attention (Module F, Topic 43) would PREVENT seeing 'future' context — incompatible with MLM's need for full bidirectional information. Hence BERT uses ONLY the Transformer ENCODER (unmasked/bidirectional self-attention).
Why can't BERT (encoder-only, bidirectional) be directly used for AUTOREGRESSIVE TEXT GENERATION the way GPT can?	BERT's bidirectional self-attention means EVERY position's representation depends on ALL OTHER positions, INCLUDING positions 'after' it. For text GENERATION (producing tokens left-to-right, one at a time), 'future' tokens DON'T EXIST YET — there's nothing for a position to bidirectionally attend to on its right side during generation. BERT was DESIGNED for tasks where the FULL INPUT IS AVAILABLE UPFRONT (classification, NER, extractive QA — 'given this full passage, classify it / find the answer span') — NOT for generating new text token-by-token, which requires the AUTOREGRESSIVE, CAUSALLY-MASKED architecture of GPT-style decoder-only models.

How does fine-tuning BERT for a downstream task (e.g., sentiment classification) typically work?

1) Start with the PRETRAINED BERT model (trained via MLM+NSP on a huge unlabeled text corpus — has learned general language understanding). 2) ADD a small task-specific 'head' on top — e.g., for classification, a single linear layer applied to the [CLS] token's final representation (a special token prepended to every input, whose final-layer representation is designed to summarize the whole sequence). 3) Continue training (fine-tune) the ENTIRE model (pretrained BERT weights + new head) on the SMALL, LABELED, task-specific dataset — typically for just a FEW epochs with a SMALL learning rate (similar to CNN transfer learning fine-tuning, Module C) — the pretrained weights provide most of the 'language understanding', the fine-tuning adapts this understanding to the specific task.

Topic 46: GPT Family — Autoregressive Language Modeling, Decoder-Only

□ Concept Overview

GPT (Generative Pre-trained Transformer) uses ONLY the Transformer DECODER (with masked/causal self-attention — no encoder, no cross-attention needed since there's no separate 'input sequence' to encode). Pretrained via standard AUTOREGRESSIVE LANGUAGE MODELING: given tokens $1 \dots t$, predict token $t+1$ — trained on this 'next token prediction' objective over massive text corpora. This SAME architecture and objective scales to GPT-2, GPT-3, GPT-4, etc. — with dramatically increasing model size and training data ('scaling').

Question	Model Answer
What is the pretraining objective for GPT, and how does it differ fundamentally from BERT's MLM?	GPT: AUTOREGRESSIVE (causal) language modeling — given tokens $x_1, \dots, x_t$, predict x_{t+1} — trained on EVERY position simultaneously (via masked self-attention, Module F) to predict the NEXT token given ONLY PREVIOUS tokens. This is a 'NATURAL' objective matching how TEXT GENERATION actually works (left-to-right). BERT's MLM: predict RANDOMLY MASKED tokens using BOTH-DIRECTION context — an 'artificial' task designed for LEARNING rich bidirectional representations, but NOT directly matching any natural generation process (you can't 'generate' by filling in random masks in a coherent order). GPT's objective DIRECTLY enables generation;

	BERT's objective DOESN'T, but provides richer bidirectional representations for understanding tasks.
Why is GPT called 'decoder-only', given that the original Transformer's decoder included cross-attention to an encoder?	The ORIGINAL Transformer decoder (Module F) has THREE components per layer: masked self-attention, encoder-decoder CROSS-ATTENTION, feedforward. GPT REMOVES the cross-attention component entirely (there's no separate encoder/input sequence to attend to — GPT processes ONE sequence: the text itself, predicting each token from previous tokens WITHIN THAT SAME sequence). What REMAINS — masked/causal self-attention + feedforward, with residuals/layer norm — is structurally a 'decoder layer minus cross-attention'. Calling it 'decoder-only' emphasizes that it retains the CAUSAL/AUTOREGRESSIVE masking property characteristic of decoders, even though there's no corresponding 'encoder' in the architecture.
[Advanced] What does 'scaling' refer to in the context of the GPT family (GPT-1 -> GPT-2 -> GPT-3 -> GPT-4), and what empirical observation motivated this strategy?	Scaling refers to INCREASING three things roughly together: MODEL SIZE (number of parameters — GPT-1: 117M, GPT-2: 1.5B, GPT-3: 175B, GPT-4: undisclosed but much larger), TRAINING DATA SIZE (amount of text the model is pretrained on), and COMPUTE (total training FLOPs). The empirical observation (formalized in 'scaling laws' research): model performance (measured by next-token-prediction loss, and downstream task performance) IMPROVES PREDICTABLY and SMOOTHLY as a function of these three factors, following roughly POWER-LAW relationships — with NO clear sign of plateauing at the scales tested. This predictability motivated the strategy of 'just scale up' (more parameters + more data + more compute) as a remarkably effective (if computationally expensive) approach to improving model capabilities, leading to the 'large language model' paradigm. Notably, scaling also revealed 'EMERGENT ABILITIES' — capabilities (e.g., few-shot learning, chain-of-thought reasoning) that appear only ABOVE certain scale thresholds, not present in smaller models trained the same way.

Topic 47: Encoder-Only vs Decoder-Only vs Encoder-Decoder (BERT vs GPT vs T5)

□ Concept Overview

Three architectural families emerged from the Transformer: ENCODER-ONLY (BERT) — bidirectional, best for UNDERSTANDING tasks (classification, NER, extractive QA). DECODER-ONLY (GPT) — autoregressive/causal, best for GENERATION tasks (text completion, chat, creative writing). ENCODER-DECODER (T5, original Transformer, BART) — combines both: encoder processes input bidirectionally, decoder generates output autoregressively while cross-attending to encoder — best for SEQUENCE-TO-SEQUENCE tasks (translation, summarization) where input and output are DISTINCT sequences.

Question	Model Answer
For each of the three architectural families (encoder-only, decoder-only, encoder-decoder), give a task each is BEST suited for, and explain WHY.	Encoder-only (BERT): SENTIMENT CLASSIFICATION — the FULL input text is available, bidirectional context helps understand the WHOLE sentence's meaning, output is a SINGLE label (not a generated sequence) — no need for autoregressive generation. Decoder-only (GPT): OPEN-ENDED TEXT GENERATION/CHATBOTS — task IS sequential generation, often with no clearly 'separate' input/output (conversation is one continuous sequence), causal structure matches the generation process directly. Encoder-decoder (T5/BART): TRANSLATION/SUMMARIZATION — there's a CLEAR DISTINCTION between INPUT (source language sentence / long document, processed BIDIRECTIONALLY by encoder for full understanding) and OUTPUT (target language sentence / summary, GENERATED autoregressively by decoder, while CROSS-ATTENDING to the encoder's full bidirectional understanding of the input).
[Advanced] Can a decoder-only model (like GPT) be used for tasks BERT excels at (e.g., classification), and vice versa? What are the practical implications?	GPT for classification: YES, via PROMPTING — frame classification as a generation task (e.g., 'Sentiment of this review: [review text]. Sentiment:' and let the model GENERATE 'positive' or 'negative' as the next token) — this works surprisingly well at SCALE (large GPT models), leveraging their broad pretraining, WITHOUT needing task-specific fine-tuning (zero/few-shot). BERT for generation: MUCH HARDER — BERT's bidirectional pretraining objective (MLM) doesn't naturally support left-to-right generation; some adaptations exist but are NOT BERT's strength. Practical implication: the INDUSTRY TREND (especially post-GPT-3) has shifted toward decoder-only models for MANY tasks (including classification via prompting) because of their GENERALITY (one model, many tasks via prompting) — though encoder-only/encoder-decoder models can still be more

PARAMETER-EFFICIENT/accurate for SPECIFIC well-defined understanding tasks when fine-tuned.

Topic 48: Fine-Tuning vs Pretraining — Transfer Learning in NLP

□ Concept Overview

PRETRAINING: train a large model on a MASSIVE, GENERAL, often UNLABELED corpus (e.g., all of Wikipedia + web text) using a SELF-SUPERVISED objective (MLM for BERT, next-token-prediction for GPT) — learns general language understanding/generation capabilities. **FINE-TUNING:** take the pretrained model, continue training on a SMALL, LABELED, TASK-SPECIFIC dataset — adapts general capabilities to the specific task. This 'pretrain then fine-tune' paradigm is the NLP analog of CNN transfer learning (Module C) — and is the dominant paradigm in modern NLP.

Question	Model Answer
Why is the 'pretrain then fine-tune' paradigm so impactful for NLP, drawing the parallel to CNN transfer learning?	Just as CNN transfer learning (Module C) leverages GENERAL VISUAL FEATURES (edges, textures) learned from a large labeled dataset (ImageNet) to bootstrap performance on smaller, task-specific datasets — NLP pretraining leverages GENERAL LANGUAGE UNDERSTANDING (grammar, semantics, world knowledge, common patterns) learned via SELF-SUPERVISED objectives on MASSIVE UNLABELED text corpora (no manual labeling needed — the 'labels' are just other parts of the text itself, e.g., the masked word or the next word). This dramatically reduces the LABELED DATA needed for any SPECIFIC downstream task — fine-tuning on even a FEW THOUSAND labeled examples can achieve strong performance, because the model isn't learning LANGUAGE from scratch, only ADAPTING its existing language understanding to the task's specific output format/labels.
What is the difference between FULL fine-tuning and PARAMETER-EFFICIENT fine-tuning (e.g., LoRA), and why does the latter matter for very large models?	Full fine-tuning: update ALL of the pretrained model's parameters during fine-tuning — for a model with billions of parameters, this requires storing GRADIENTS and OPTIMIZER STATES for ALL parameters (memory-intensive — often several times the model size), and produces a SEPARATE FULL COPY of the model for EACH fine-tuned task. Parameter-efficient fine-tuning

	(e.g., LoRA - Low-Rank Adaptation): FREEZE the pretrained weights, and add SMALL, NEW, TRAINABLE LOW-RANK matrices ALONGSIDE existing weight matrices — only these small additions are updated during fine-tuning (often <1% of total parameters). This DRAMATICALLY reduces memory requirements for fine-tuning, and allows STORING/SWITCHING BETWEEN many task-specific 'adapters' (the small LoRA matrices) while sharing ONE COPY of the large frozen base model — critical for making fine-tuning of multi-billion-parameter models PRACTICAL on limited hardware.
[Advanced] What is 'catastrophic forgetting' in the context of fine-tuning, and how does it relate to learning rate choices (connecting back to Module B/C)?	Catastrophic forgetting: during fine-tuning on a NEW, often NARROW task, the model's weights shift AWAY from their pretrained values in a way that DEGRADES the GENERAL capabilities learned during pretraining — the model 'forgets' general knowledge while 'overfitting' to the narrow fine-tuning task. This is MORE likely with: LARGE learning rates (large weight updates that drastically alter pretrained representations — same concern as discussed for CNN fine-tuning in Module C), MANY fine-tuning epochs/steps on a SMALL dataset (more opportunity to drift/overfit), and FULL fine-tuning (vs parameter-efficient methods, where the FROZEN base weights inherently CANNOT 'forget' since they're never updated). Mitigations: small learning rates, fewer epochs, early stopping on a held-out set that includes some GENERAL capability checks (not just the narrow task), and parameter-efficient methods (LoRA) which structurally limit how much the underlying representations can shift.

Topic 49: Tokenization for Transformers (BPE, WordPiece, SentencePiece)

□ Concept Overview

Transformers operate on a FIXED VOCABULARY of tokens — but natural language has an UNBOUNDED vocabulary (new words, typos, rare words). SUBWORD TOKENIZATION solves this: common words remain whole tokens, but RARE/UNKNOWN words are SPLIT into smaller SUBWORD UNITS (or even characters) — guaranteeing NO out-of-vocabulary problem (any string can be represented as SOME sequence of subword tokens), while keeping the vocabulary size MANAGEABLE (e.g., 30,000-50,000 tokens). BPE (Byte-Pair Encoding), WordPiece (BERT), and SentencePiece (T5, many multilingual models) are the dominant algorithms, all building vocabularies by ITERATIVELY MERGING frequent character/subword pairs.

Question	Model Answer
What problem does subword tokenization solve compared to word-level tokenization (recall the OOV problem from Word2Vec in earlier modules)?	Word-level tokenization: vocabulary = set of all words seen in training — ANY word NOT in this vocabulary at inference time (typos, new words, rare technical terms, names) becomes an UNKNOWN/OOV token, LOSING all information about that word (exactly the OOV problem discussed for Word2Vec). Subword tokenization: builds a vocabulary of COMMON SUBWORD UNITS (e.g., 'un', 'happi', 'ness', plus common whole words) — an unseen word like 'unhappiness' can be REPRESENTED as a SEQUENCE of KNOWN subwords ['un','happi','ness'] — NO information is lost to an 'unknown' token, EVERY possible string can be tokenized (in the worst case, falling back to individual CHARACTERS, which are always in the vocabulary), completely eliminating the OOV problem.
Describe the high-level algorithm behind Byte-Pair Encoding (BPE) vocabulary construction.	1) START with a vocabulary of INDIVIDUAL CHARACTERS (every character that appears in the training corpus). 2) COUNT the frequency of every ADJACENT PAIR of tokens (initially, pairs of characters) across the corpus. 3) MERGE the MOST FREQUENT pair into a SINGLE NEW TOKEN, add it to the vocabulary. 4) REPEAT steps 2-3 for a fixed number of iterations (determining the final vocabulary size, e.g., 30,000 merges) — each iteration merges the currently-most-frequent ADJACENT pair (which may now include previously-merged multi-character tokens). Result: COMMON words/word-parts (frequently occurring sequences) end up as SINGLE tokens, while RARE words remain split into SMALLER (more common) subword pieces or individual characters.
[Advanced] How does the choice of vocabulary SIZE in subword tokenization create a tradeoff, and what are the implications for sequence length / model context window?	LARGER vocabulary (e.g., 100,000 tokens): MORE words/common phrases are represented as SINGLE tokens — SHORTER token sequences for the same text (fewer tokens needed to represent a sentence), but the embedding/output layers (which scale with vocabulary size) become LARGER (more parameters), and RARE tokens in this larger vocabulary may have LESS TRAINING SIGNAL (seen fewer times) for their embeddings to be well-learned. SMALLER vocabulary (e.g., 10,000 tokens): MORE words get split into MULTIPLE subword tokens — LONGER token sequences for the same text. Implication for context window:

a Transformer's self-attention has $O(n^2)$ cost in sequence length n (Module E/F) — a SMALLER vocabulary means the SAME TEXT consumes MORE of the model's fixed 'context window' (in tokens), effectively REDUCING how much actual TEXT the model can process at once, and INCREASING computational cost for the same text. This is why vocabulary size (typically 30K-100K+ for modern LLMs) is a carefully tuned design choice balancing these factors.

Topic 50: Vision Transformers (ViT) — Applying Transformers to Images

□ Concept Overview

ViT (Vision Transformer) applies the Transformer architecture (originally for NLP sequences) to IMAGES by: splitting the image into FIXED-SIZE PATCHES (e.g., 16x16 pixels), FLATTENING and LINEARLY PROJECTING each patch into an embedding vector (analogous to a 'word' token), adding POSITIONAL ENCODINGS (since, like text, patches have no inherent order to a Transformer), and feeding this SEQUENCE OF PATCH EMBEDDINGS through a standard Transformer ENCODER (BERT-style) — with a [CLS] token for classification. Demonstrates that SELF-ATTENTION can REPLACE CONVOLUTIONS entirely, given enough training data.

Question	Model Answer
How does ViT convert an image into a 'sequence' that a Transformer can process?	An image (e.g., 224x224 pixels) is divided into a GRID of FIXED-SIZE non-overlapping PATCHES (e.g., 16x16 pixels each, giving a 14x14=196 grid of patches). Each patch (16x16x3 = 768 values, flattened) is LINEARLY PROJECTED (via a learned matrix, similar to a word embedding lookup) into a fixed-dimensional embedding vector — this 'patch embedding' is the Transformer's analog of a 'token embedding' in NLP. The 196 patch embeddings form a SEQUENCE (analogous to a sequence of 196 'words'), to which POSITIONAL ENCODINGS are added (since, like word order, PATCH POSITION in the image matters and isn't inherently captured by self-attention) — this sequence is then fed through a standard Transformer encoder, exactly as BERT processes a sequence of word tokens.
What is the key empirical finding/tradeoff of ViT compared to CNNs, related to the 'inductive bias' concept?	CNNs have STRONG BUILT-IN INDUCTIVE BIASES for images: LOCALITY (convolution filters only look at small local regions — Module C) and TRANSLATION EQUIVARIANCE (same

filter applied everywhere). These biases mean CNNs can learn EFFECTIVELY from RELATIVELY SMALL datasets — the architecture itself 'encodes' useful assumptions about images. ViT has MUCH WEAKER inductive biases (self-attention can, in principle, attend GLOBALLY from layer 1, with no inherent locality assumption — positional encodings provide SOME spatial information but it's LEARNED, not hard-coded like convolution). Key finding: ViT trained on SMALL/MEDIUM datasets UNDERPERFORMS comparable CNNs (lacks the helpful biases) — but trained on VERY LARGE datasets (hundreds of millions of images), ViT MATCHES OR EXCEEDS CNNs — the model can LEARN useful spatial patterns/biases FROM DATA, given enough of it, rather than needing them HARD-CODED into the architecture. This mirrors the broader 'scale can substitute for inductive bias, given enough data' theme seen across modern deep learning.

Topic 51: Scaling Laws & Large Language Models (Overview)

□ Concept Overview

Scaling laws describe EMPIRICAL, often POWER-LAW relationships between a model's PERFORMANCE (typically, pretraining loss) and three factors: MODEL SIZE (parameters), DATASET SIZE (tokens), and COMPUTE (FLOPs). Key finding (Chinchilla, Kaplan et al.): for a FIXED compute budget, there's an OPTIMAL BALANCE between model size and data size — many earlier large models were UNDERTRAINED relative to their size (too big for the data they were trained on). 'Emergent abilities' — capabilities appearing only above certain scale thresholds — are an active area of study and debate.

Question	Model Answer
What do 'scaling laws' describe, and why are they useful for planning large model training runs?	Scaling laws are EMPIRICALLY-FIT (often power-law) relationships: $\text{Loss} \approx f(N, D, C)$, where N =number of parameters, D =dataset size (tokens), C =compute (FLOPs, roughly proportional to $N \cdot D$ for transformer training). USEFULNESS: training a multi-billion-parameter model is EXTREMELY EXPENSIVE (millions of dollars) — scaling laws, FIT on SMALLER, CHEAPER experiments (e.g., training models from 10M to 1B parameters), allow EXTRAPOLATING/PREDICTING the

	performance of a MUCH LARGER model BEFORE actually training it — informing decisions like 'given a \$10M compute budget, what's the OPTIMAL combination of model size and data size to minimize loss?' WITHOUT needing to run many full-scale training experiments.
What was the key finding of the 'Chinchilla' scaling laws regarding the relationship between model size and dataset size?	Earlier large models (e.g., GPT-3) were, according to Chinchilla's analysis, RELATIVELY UNDERTRAINED — TOO LARGE (many parameters) relative to the amount of TRAINING DATA used, given the COMPUTE BUDGET spent. Chinchilla's finding: for a FIXED compute budget, model size and dataset size should be scaled ROUGHLY EQUALLY (both increased proportionally) — a SMALLER model trained on MORE DATA (with the same total compute) can OUTPERFORM a LARGER model trained on LESS DATA. This led to a shift in subsequent model development toward 'compute-optimal' training — choosing model sizes that are smaller than previous trends might suggest, but training them on PROPORTIONALLY MORE DATA.
[Advanced] What are 'emergent abilities' in LLMs, and why is their existence/interpretation somewhat debated?	Emergent abilities: capabilities (e.g., multi-step arithmetic, chain-of-thought reasoning, in-context few-shot learning) that appear to be ESSENTIALLY ABSENT (near-random performance) in SMALLER models, but appear SUDDENLY/SHARPLY once model scale crosses some threshold — i.e., performance on these tasks is NOT a smooth/predictable extrapolation from smaller-scale results, UNLIKE the smooth power-law scaling of PRETRAINING LOSS itself. DEBATE: some researchers argue these 'sudden jumps' are partly an ARTIFACT OF THE EVALUATION METRIC (e.g., EXACT-MATCH accuracy on multi-step tasks can look like a 'sudden jump' even if the model's UNDERLYING per-step error rate is improving SMOOTHLY — a smoothly improving per-step accuracy can translate to a SHARP jump in 'all steps correct' accuracy once per-step error drops below a threshold) — i.e., the underlying CAPABILITY may be improving smoothly, but the CHOSEN METRIC makes it LOOK discontinuous/emergent. This remains an active area of research with implications for how predictable/controllable future capability gains from scaling will be.

Master Questions — Connecting the Entire Deep Learning Journey

Connecting All Modules — Synthesis & Interview-Style Reasoning

Question	Model Answer
Trace the complete evolution from a single perceptron to a Transformer, explaining what problem each major innovation solved.	Perceptron (linear, single layer) -> couldn't solve XOR (non-linear separability) -> MLP with hidden layers + non-linear activations solved this, but suffered vanishing gradients with depth -> ReLU + better initialization (He/Xavier) + BatchNorm + residual connections (ResNet) enabled training very deep networks -> CNNs added spatial inductive biases (parameter sharing, locality) for images, RNNs added temporal recurrence for sequences -> RNNs suffered vanishing gradients over LONG sequences -> LSTM/GRU gating (additive cell state) mitigated this for moderate lengths -> Seq2Seq enabled variable-length input/output but introduced a fixed context-vector BOTTLENECK -> Attention solved the bottleneck by letting the decoder dynamically access ALL encoder states -> Self-attention generalized this WITHIN a single sequence, providing constant-distance connections between any two positions AND full parallelization -> Transformer built an architecture ENTIRELY from self-attention + feedforward + residuals/layernorm, removing recurrence entirely -> BERT (encoder-only, bidirectional, MLM) and GPT (decoder-only, autoregressive) became the two dominant pretraining paradigms -> Scaling (more parameters+data+compute, guided by scaling laws) became the dominant strategy for improving capabilities, leading to modern LLMs.
How do residual connections (ResNet, Module C) and the additive cell-state update (LSTM, Module D) and Transformer residual connections (Module F) all address a CONCEPTUALLY SIMILAR problem, despite being in different architectures?	All three address VARIANTS of the VANISHING GRADIENT / INFORMATION DECAY problem that arises when information must pass through MANY repeated transformations (depth in CNNs, time steps in RNNs, layers in Transformers) — repeated MULTIPLICATIVE transformations cause exponential decay. Each solution introduces an ADDITIVE 'shortcut' pathway: ResNet's $\text{output} = F(x) + x$ lets gradients skip directly to earlier LAYERS. LSTM's $C_t = f_t * C_{t-1} + i_t * \text{candidate}$ lets information persist across TIME STEPS with minimal multiplicative decay (when $f_t \approx 1$, $i_t \approx 0$). Transformer residuals ($\text{output} = x + \text{Sublayer}(x)$), used around BOTH attention and feedforward sub-layers in every

	layer) let gradients flow through very deep STACKS of Transformer layers. The SHARED PRINCIPLE: provide an 'easy default' (identity/pass-through) path that the network can fall back to, with the 'hard' transformation (F, gates, sublayer) only needing to learn the DEVIATION from this default — making optimization dramatically easier in deep/long architectures.
Compare and contrast how CNNs (Module C) and Transformers (Module F) each introduce useful 'inductive biases' for their respective domains (images, sequences) — and how ViT (Module G) relates the two.	CNNs encode LOCALITY (each output depends on a small spatial neighborhood, via small filters) and TRANSLATION EQUIVARIANCE (same filter applied everywhere via parameter sharing) — strong, HARD-CODED assumptions well-suited to images (nearby pixels are related; a pattern like 'edge' means the same thing anywhere in the image). Transformers' self-attention has essentially NO locality bias — any position can attend to any other position with EQUAL EASE (modulo positional encodings) — a WEAKER, more FLEXIBLE bias, requiring positional encodings to inject ANY notion of order/position at all. ViT APPLIES the weak-bias Transformer architecture to images (normally a CNN's strong-bias domain) — the empirical result (ViT underperforms CNNs on small data, matches/exceeds on huge data) illustrates a general principle: STRONG inductive biases help with LIMITED data (the architecture 'knows' useful assumptions a priori) but can become a CEILING with abundant data (the bias might be SLIGHTLY WRONG or SUBOPTIMAL compared to what could be LEARNED from data) — WEAK-bias, FLEXIBLE architectures (Transformers) can SURPASS strong-bias architectures (CNNs) GIVEN ENOUGH DATA to learn the relevant structure from scratch.
[Advanced] An interviewer asks: 'Why did it take until 2017 (Transformers) for an architecture without recurrence to dominate NLP, given that the MATHEMATICAL CONCEPT of attention/weighted-averaging is quite simple?' How would you answer, synthesizing concepts from across this curriculum?	Several factors had to CONVERGE: 1) COMPUTE: self-attention's $O(n^2)$ cost and massive parallelizability only became PRACTICALLY ADVANTAGEOUS once GPUs/TPUs with sufficient PARALLEL THROUGHPUT and MEMORY became widely available (mid-2010s) — earlier, RNNs' sequential-but-cheaper-per-step computation may have been more practical. 2) ARCHITECTURAL PREREQUISITES: training a PURELY attention+feedforward stack DEEP enough to be competitive REQUIRED the residual connections + layer normalization techniques (refined through CNN research like ResNet, ~2015) to manage vanishing gradients in deep stacks WITHOUT recurrence's (partial)

mitigations. 3) DATA AVAILABILITY: the SELF-SUPERVISED pretraining paradigms (MLM, autoregressive LM) that made Transformers so impactful required LARGE TEXT CORPORA + the COMPUTE to pretrain on them — both became increasingly available through the 2010s. 4) The SPECIFIC FORMULATION (scaled dot-product, multi-head, positional encodings) required iterating on EARLIER attention work (Bahdanau/Luong, 2014-2015, which was ITSELF layered ON TOP OF RNN Seq2Seq) — the 2017 Transformer paper's key insight was REMOVING the RNN entirely while RETAINING (and refining) the attention mechanism, a conceptual leap that built on several years of incremental attention research. In short: the IDEA (weighted averaging/attention) was simple and existed earlier in a more limited form (RNN+attention hybrids), but COMPUTE, ARCHITECTURAL TRAINING TECHNIQUES, and DATA all had to reach a certain maturity for a PURELY attention-based architecture to be BOTH TRAINABLE and ADVANTAGEOUS COMPARED TO the RNN-based alternatives of the time.

How would you explain, end-to-end, what happens when you type a prompt into a GPT-style chatbot and it generates a response — using terminology from EVERY module of this curriculum (A through G)?

1) TOKENIZATION (G): your prompt text is split into SUBWORD TOKENS (BPE/WordPiece). 2) EMBEDDING + POSITIONAL ENCODING (F): each token is converted to a vector, with POSITION information added (since self-attention is order-agnostic). 3) DECODER-ONLY TRANSFORMER FORWARD PASS (F, G): the token sequence passes through MANY STACKED layers, each containing MASKED MULTI-HEAD SELF-ATTENTION (every token attends to PREVIOUS tokens — Module E/F concepts) and POSITION-WISE FEEDFORWARD networks (Module F), with RESIDUAL CONNECTIONS + LAYER NORM (Modules B/C/F) enabling this VERY DEEP stack to train/run stably. 4) OUTPUT: the final layer produces a probability distribution (via softmax — Module B's cross-entropy connection) over the ENTIRE VOCABULARY for the NEXT TOKEN. 5) SAMPLING: a token is selected from this distribution (greedy/sampling strategies). 6) AUTOREGRESSIVE LOOP (D, F): this generated token is APPENDED to the sequence, and steps 2-5 REPEAT — generating one token at a time (the SEQUENTIAL bottleneck of autoregressive generation, Module F Topic 44) — until an end-of-sequence token or length limit is reached. 7) UNDERLYING ALL OF THIS: the model's WEIGHTS were learned via

	PRETRAINING (autoregressive language modeling, Module G — itself trained via BACKPROPAGATION/chain rule, Module B, using ADAM/AdamW optimizers, Module B) on a MASSIVE corpus, GUIDED by SCALING LAWS (G) determining the model's size/data tradeoffs, and potentially FINE-TUNED (G) on instruction-following/conversational data to produce helpful chat-style responses.
--	--

End of Deep Learning Question Bank — From Perceptron to Transformers